Professional Expertise Distilled

Team Foundation Server 2013 Customization

Learn how to customize TFS and create useful plugins for your organization

Gordon Beeming

[PACKT] PUBLISHING enterprise professional expertise distilled

www.allitebooks.com

Team Foundation Server 2013 Customization

Learn how to customize TFS and create useful plugins for your organization

Gordon Beeming

[PACKT] PUBLISHING enterprise professional expertise distilled

BIRMINGHAM - MUMBAI

www.allitebooks.com

Team Foundation Server 2013 Customization

Copyright © 2014 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: January 2014

Production Reference: 1140114

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78217-714-2

www.packtpub.com

Cover Image by Duraid Fatouhi (duraidfatouhi@yahoo.com)

www.allitebooks.com

Credits

Author
Gordon Beeming

Reviewers
Chandana N. Athauda
Logutov Michael
Mathias Olausson
Jim Szubryt

Acquisition Editors
Antony Lowe
Llewellyn Rozario

Lead Technical Editor
Deepika Singh

Technical Editors
Akashdeep Kundu
Mrunmayee Patil

Copy Editors
Roshni Banerjee
Mradula Hegde

Project Coordinator
Jomin Varghese

Proofreader
Maria Gould

Indexer
Mehreen Deshmukh

Production Coordinator
Saiprasad Kadam

Cover Work
Saiprasad Kadam

www.allitebooks.com

About the Author

Gordon Beeming is an energetic and passionate person who always strives to find ways to increase and improve the productivity and friendliness of the systems he works with. He has been a software developer for a little over five years. In this time, he has also gained strong experience in TFS and is a part of the ALM Rangers. He currently works for Derivco, which is a very unique company to work for, based in Durban, South Africa.

www.allitebooks.com

About the Reviewers

Chandana N. Athauda is currently employed at Brunei Accenture Group (BAG) Networks, Brunei, and serves as a technical consultant focusing on adopting new technologies toward solid solutions. He has been working professionally in the IT industry for more than 12 years (ex-Microsoft Most Valuable Professional (MVP) and Microsoft Ranger for TFS). His roles in the IT industry have spanned the entire spectrum from programmer to technical consultant. Technology has always been a passion for him. In his spare time, he enjoys watching association football and of course, Crossfit WOD-ing.

If you would like to talk to Chandana about this book, feel free to write to him at `info@inzeek.net` or tweet him `@inzeek`.

Logutov Michael is a professional web developer working in Moscow. He is also the administrator of the company's Team Foundation Server and has been customizing it since 2007.

Mathias Olausson is the CEO at Solidify AB, specializing in software craftsmanship and application lifecycle management. With close to 20 years of experience as a software consultant and trainer, he has worked in numerous projects and organizations, which has been very valuable, while using Visual Studio as a tool for improving the way we build software. Olausson has been a Microsoft Visual Studio ALM MVP for six years. He is also active as a Visual Studio ALM Ranger, and most recently in the role of project lead for the Visual Studio Lab Management Guide project. He is a frequent speaker on Visual Studio and Team Foundation Server at conferences and industry events, and he blogs at `http://msmvps.com/blogs/molausson`. He has worked on the following two books:

www.allitebooks.com

- *Pro Application Lifecycle Management with Visual Studio 2012* (*APress*, 1430243449) which can be found at `http://www.amazon.com/Application-Lifecycle-Management-Visual-Professional/dp/1430243449/`
- *Pro Team Foundation Service* which can be found at `http://www.amazon.com/Team-Foundation-Service-Experts-Voice-ebook/dp/B00FGSVJ92`

Jim Szubryt has been working in the ALM space for the last seven years. He is a Microsoft Application Lifecycle Management (ALM) MVP and Microsoft ALM Ranger. He has been working with TFS since 2006 for companies with as few as five developers and as many as 2,500. He is an Application Tech Arch Manager in the Enterprise Workforce with Accenture.

I'd like thank my wife Sue, daughters Camille and Ari, and son Austin, for their patience with me during the holidays while working on this.

---

www.allitebooks.com

www.PacktPub.com

Support files, eBooks, discount offers, and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print, and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Instant Updates on New Packt Books

Get notified! Find out when new books are published by following `@PacktEnterprise` on Twitter, or the *Packt Enterprise* Facebook page.

www.allitebooks.com

www.allitebooks.com

Table of Contents

Preface **1**

Chapter 1: Customizing the TFS Process Templates for Your Business **5**

Prerequisites or tools **5**

Customizing the TFS work items **6**

Adding a new field using the TFS Power Tools 6

Adding a field to the work item form layout using the TFS Power Tools 10

Importing a WITD using the TFS Power Tools 11

Adding a new field using the raw .xml files 11

Adding a field to the work item form layout using the raw .xml files 12

Altering the work item workflow using the raw .xml files 12

Importing a WITD using the raw .xml files 13

Adding a new portfolio to the process template **14**

Creating a new work item type definition 14

Adding a new portfolio backlog 14

Adding a new work item category 15

Importing a new work item portfolio 15

Importing the work item type definition 16

Importing the categories file 16

Importing the process configuration file 16

Summary **17**

Chapter 2: Increasing Web Access Usability **19**

Customizing TFS dashboard tiles **19**

Adding a tile to the dashboard 20

Pinning build definitions to the team dashboard 22

Pinning recent changes to the dashboard 23

Changing the default columns for portfolio boards **24**

Adding fields to the add panels **25**

www.allitebooks.com

Creating a basic JavaScript plugin **26**
Getting started with an empty plugin 27
Importing a plugin into TFS Web Access 28
Setting up the debug mode 29
Enabling the Script Debug Mode 30
Configuring a Fiddler AutoResponder rule 30
Adding information to display work items 31
Summary **35**
Chapter 3: Enhancing Work Items with Custom Controls **37**
Creating a Windows Forms control **38**
Writing the code 38
Deploying the code 40
Debugging the code 41
Creating a control for use inside the Web Access **41**
Writing the code 41
Deploying and debugging the code 43
Using the control on an existing work item definition **44**
Displaying a web page with rich data inside a WITD **46**
Summary **49**
Chapter 4: Creating Server Plugins **51**
Using a TFS plugin over a check-in policy **51**
The basics of a server plugin **52**
Setting up the code 52
Deploying server plugins 53
Debugging server plugins 53
Creating a check-in plugin **54**
Creating a work item changed plugin **57**
Summary **60**
Chapter 5: Creating TFS Scheduled Jobs **61**
Creating a TFS job **62**
Writing the code 62
Deploying the code 64
Registering the custom job and adding it to a schedule 64
Verify that the job is installed 65
Checking the job history **65**
Debugging a TFS job **66**
Deregistering the custom job with TFS **67**
Summary **68**

Chapter 6: Customizing a TFS Build Process **69**
Creating a copy of the default build template **70**
Downloading the default template 70
Using the template in a build 71
Creating a custom build activity **71**
Creating the custom activity project 72
Zipping the current build 74
Uploading the ZIP file to FTP 74
Sharing the build number and URL with the ZIP file 75
Adding a custom activity to an existing build template **76**
Creating a build template project 76
Adding a build template argument **79**
Summary **82**
Index **83**

Preface

Team Foundation Server is an Application Lifecycle Management (ALM) tool that helps you manage and plan all your work, including portfolio management, management of your test cases, and test execution. You are able to interact with TFS using various clients, such as Microsoft Visual Studio and Microsoft Test Manager (MTM), and a web portal. This book will show you how to customize TFS using lots of customization and extensibility points. After going through this book, you will have a good understanding of how to help your company by enhancing the user experience and productivity of users while using TFS.

What this book covers

Chapter 1, *Customizing the TFS Process Templates for Your Business*, covers customizing the TFS process template by adding new fields to an existing work item type using the TFS Power Tools and by editing raw XML files. We will also create a new work item portfolio.

Chapter 2, *Increasing Web Access Usability*, covers customizing the TFS Web Access by lighting up the dashboard with useful information, and we will also create a web extension using JavaScript to light up the task and portfolio boards.

Chapter 3, *Enhancing Work Items with Custom Controls*, covers creating custom controls that can be used on work items to extend the way in which we interact with the data in our work items.

Chapter 4, *Creating Server Plugins*, covers creating some server plugins to help add some business rule validations to TFS check-ins and work item data changes.

Chapter 5, *Creating TFS Scheduled Jobs*, covers TFS scheduled jobs, and we will learn how to create a new job, register it, deregister it, and check the history of running scheduled jobs.

Chapter 6, *Customizing a TFS Build Process*, covers customizing the TFS build process and then we set up a scenario for open source projects, where we set up a CI build that tweets new releases and locations to get the new bits.

What you need for this book

For this book, you will need the following software:

- Team Foundation Server 2013 RTM
- Visual Studio 2013 RTM
- TFS Power Tools 2013

Who this book is for

This book is for all users who need or want to customize TFS to make their organization more productive while using TFS. Users will need an understanding of the terms used in TFS to easily navigate through the book.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles are shown as follows: "We will check all the submitted items to see if any of them are in the `Framework` folder."

A block of code is set as follows:

```
using System.Diagnostics;
using Microsoft.TeamFoundation.Framework.Server;
using Microsoft.TeamFoundation.VersionControl.Server;
```

Any command-line input or output is written as follows:

```
cd C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem
Tracking\TypeDefinitions
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "This will perform the deployment locally to your machine each time you build. To enable easy debugging, switch to the **Debug** tab, click on the **Start external program** radio button, and click on the **browse** button."

Warnings or important notes appear in a box like this.

Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1

Customizing the TFS Process Templates for Your Business

In this chapter, we will be looking at extending the TFS process template in the following ways:

- Customizing the TFS work items using the TFS Power Tools
- Customizing the TFS work items using raw `.xml` files
- Adding a new portfolio to an existing process template

This chapter will assume that you have downloaded the **Microsoft Visual Studio Scrum 2013** process template using Visual Studio and saved it to the location `C:\ProcessTemplates\`. You can see more information on how to do this by going to `http://msdn.microsoft.com/en-us/library/vstudio/ff452587.aspx`.

Prerequisites or tools

To complete all of the examples in this chapter, you will require Visual Studio Professional or higher with TFS Power Tools installed. You can download and install the latest version of TFS Power Tools from `http://visualstudiogallery.msdn.microsoft.com/f017b10c-02b4-4d6d-9845-58a06545627f`. If you do not wish to perform the customizations using the Process Template Editor in Power Tools, you can use Visual Studio Team Explorer and higher. The Power Tools are a set of enhancements, tools, and command-line utilities that increase the productivity of the Team Foundation Server scenarios; the version used at the time of writing this chapter was 12.0.60919.00.

Customizing the TFS work items

Although TFS has many fields that cover a lot of common pieces of data about a work item that teams would want to capture, you often find yourself requiring more fields. In this section, we will be adding two new fields to the Product Backlog item **Work Item Type Definition** (**WITD**) at `http://msdn.microsoft.com/en-us/library/ms243932.aspx`. First, we will add a field using the TFS Power Tools; and then we will add another field using the raw `.xml` files without using any of the Power Tools GUI.

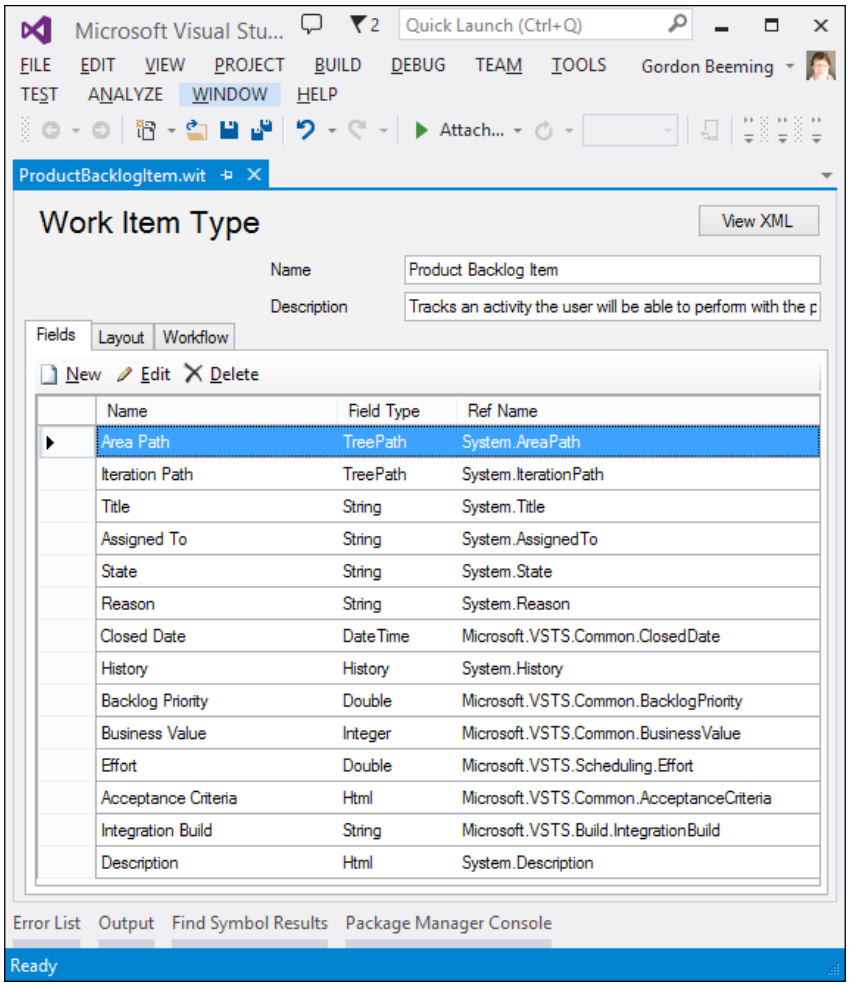
Adding a new field using the TFS Power Tools

In the example of adding a field with the TFS Power Tools, we will be adding a field for **Stream of Work**. This field will be a text field that is restricted to the options **Website**, **Email**, and **Phone Call**, and it will be a required field.

We will notice that the following points are important in the creation of this new field:

- **Stream of Work**: This will be the name of the field
- **Text Field**: This field will be a string field type, so it will support the expected data type
- **Restricted options**: This field will make use of the `ALLOWEDVALUES` element within the field rules, with the allowed values such as Website, Email, and Phone Call
- **Required**: This field will need to make use of the `Required` element in the field rules

To get started with our new field, open the `ProductBacklogItem.xml` WITD inside Visual Studio 2013 with TFS Power Tools installed, which you should be able to locate in the path `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\TypeDefinitions`. When it is open, you should see the following screenshot in your IDE:

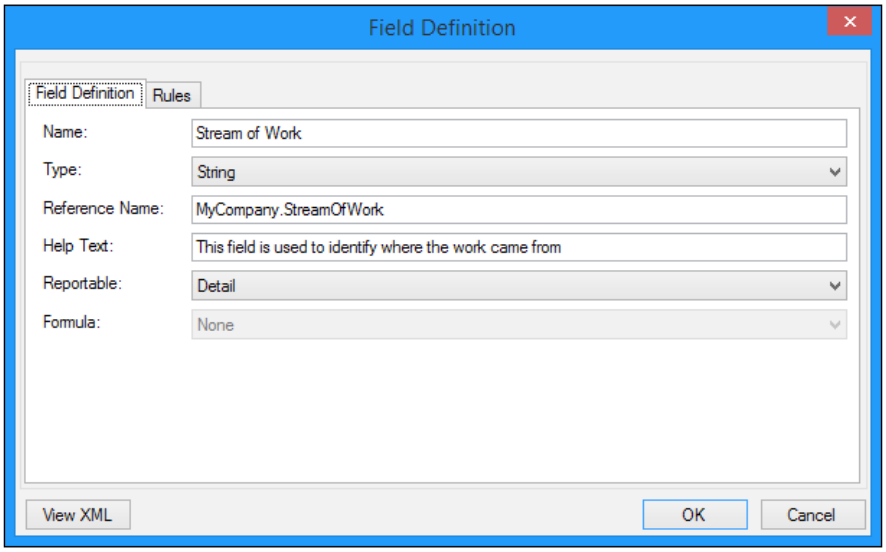


ProductBackLog.xml opened using the TFS Power Tools

Click on the **New** button. This will open up the **Fields Definition** window. The first tab contains information that is used to identify this field, such as its name, type, whether it is reportable, and help text that is used in the tooltip for the field. The second tab contains all the rules for this tool that are used to determine how this field behaves during the work items workflow.

www.allitebooks.com

We are going to use 2 bits from the 4 bits of information that we gathered from our fields' requirements for the details of this field. We are going to use `Stream of Work` for the **Name** and **Reference Name** fields, and then also Text Field which we will translate to being a string data type for this field. Populate the form with the values shown in the following screenshot:



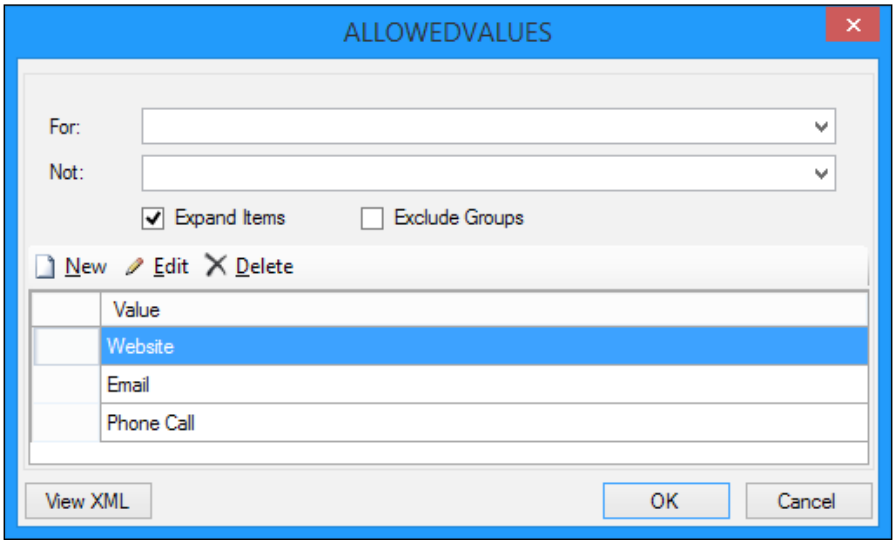
Field Definition details for Stream of Work

It would be a good idea to have some sort of naming convention for the name and reference name of fields, as it's very easy to make a mess with random new fields in TFS. The reference name determines many usages of the field in TFS. For example, the reference name replacing periods (.) with underscores (_) will give you the name of the field in the warehouse if the field is configured to be **Reportable**, and it is also used to reference this field if it is needed in other parts of the process template. The name values will be used in many other places. For example, when you use the query editor, the names will appear in the fields list as well as in the column header in the grids that display the field.

> The field definition name and reference name are unique. If combined with a type other than what's used in other Work Item Definitions (in another collection in the same TFS instance), it can cause your reporting warehouse to stop refreshing. This is because the two different collection definitions will be conflicting, and the refresh job would throw an error.

For more guidelines on *Naming conventions for work item tracking objects*, go to `http://msdn.microsoft.com/en-us/library/ms194941.aspx`.

Now, switch to the **Rules** tab so that we can define the rest of our new field. We are going to add the remaining items from our requirement now. These items are the ones in which the field must have a restricted set of options and must be required. To do this, click on **New**, then select **ALLOWEDVALUES** from the list, and then click on **OK**. Then click on **New** and enter the value `Website`. Repeat this step for `Email` and `Phone Call`. Your form should look like the window in the following screenshot:



ALLOWEDVALUES options

Next, click on **New**, select **REQUIRED**, and double-click on **OK**. Your rules will look as follows:

Rules for the Stream of Work field

After you click on **OK**, the new field will be added to the Product Backlog Item WITD. To find out more about the different rules and elements that you can use with a field, see `http://msdn.microsoft.com/en-us/library/aa337627(v=vs.90).aspx`.

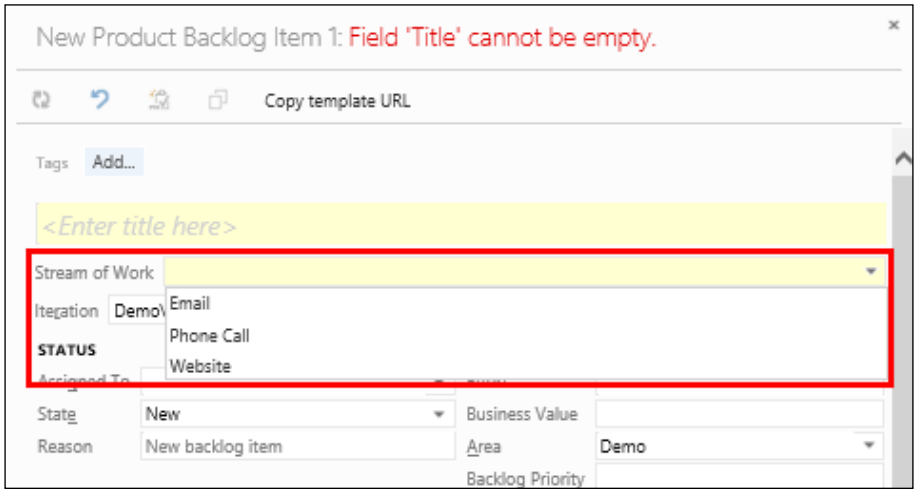
Adding a field to the work item form layout using the TFS Power Tools

Switch to the **Layout** tab. We will now add this control to the form that is displayed for this work item type. Right click on the first column node in the tree view and click on **New Control**. In the column options that appear on the right, locate the **Label** field and type in `Stream of Work`. Then, select **MyCompany.StreamOfWork** in **Field Name**. You can use the **Preview Form** button to preview an example of the layout of the controls.

Importing a WITD using the TFS Power Tools

That's all that is required to add a new field. To upload this work item type back into TFS, go to **Tools | Process Editor | Work Item Types | Import WIT**. Then, select your **Team Project Collection** and click on **OK**. Click on **BROWSE** and browse for the `ProductBacklogItem.xml` file that you have just modified, then select the Team Project you would like to upload this change to, and finally click on **OK**.

If you create a new Product Backlog Item in the Web Access, you will notice that your change is visible.



Stream of Work in the new Product Backlog Item form

Notice how all the values are listed alphabetically. TFS will automatically display all list items alphabetically.

Adding a new field using the raw .xml files

In the example of adding a field by editing the `.xml` file, we will be adding a field for Product Code. This field will be a number that will be used to highlight a product code that the work item is related to (if applicable). When any work item for that product code is approved, the product code can't be changed.

We will notice that the following points are important for the creation of this new field:

- **Product Code**: This will be the name of the field
- **Number**: This field will be an integer field type, so it supports the expected data type
- **When work is approved, the value can't change**: This will mean that when a PBI moves into the approved state, we need to lock the value of the product code

To get started with our new field, open the `ProductBacklogItem.xml` WITD inside your favorite text editor. You should be able to locate it in the path `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\TypeDefinitions`.

In the `.xml` file, find the end of the **FIELDS** node. Just before it, insert the XML shown as follows:

```
<FIELD name="Product Code" refname="MyCompany.ProductCode"
type="Integer" reportable="dimension" />
```

We have now defined the name, reference name, and data type for this field.

Adding a field to the work item form layout using the raw .xml files

Next, we need to add the field to the work item layout. Locate the control we added earlier in the chapter, and add a new control below it with the following XML:

```
<Control FieldName="MyCompany.ProductCode" Type="FieldControl"
Label="Product Code" LabelPosition="Left" />
```

We have now added the field to the layout with a label Product Code.

Altering the work item workflow using the raw .xml files

The next step of meeting the requirements is to make sure that this field is locked when the state is changed to `Approved`. Find the Approve State in the XML, look for `<STATE value="Approved">`, and add the following field rule to the `FIELD` element:

```
<FIELD refname="MyCompany.ProductCode">
  <FROZEN />
</FIELD>
```

Our new field will now be locked (frozen) when the state of a PBI changes to `Approved` for any transition that will be approved. If we only want this to apply from a certain state to be approved, we would add this rule to the transition between the two states.

Importing a WITD using the raw .xml files

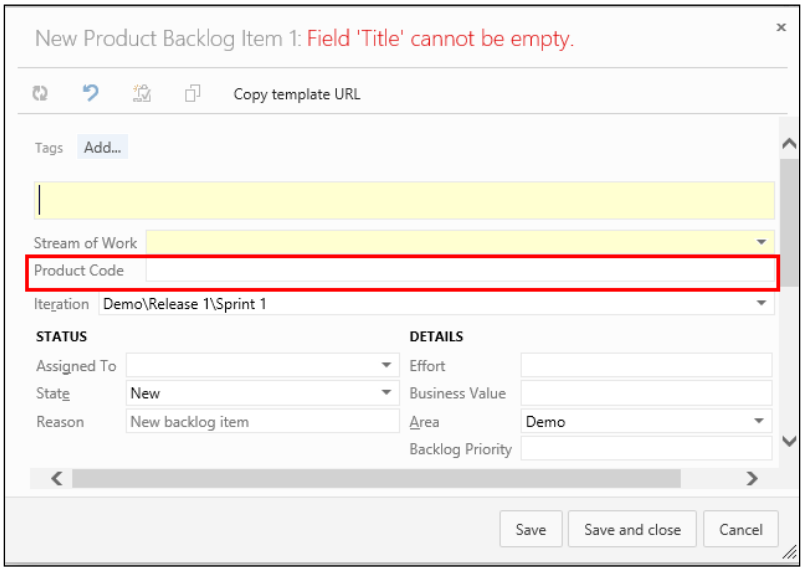
The last step will be importing this definition into TFS. Open a **Developer CMD console**, and then we will use the `witadmin` tool to import this definition as follows:

```
cd C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem
Tracking\TypeDefinitions
```

```
witadmin importwitd /collection:http://localhost:8080/tfs /p:Demo
/f:ProductBacklogitem.xml
```

You should notice a message that reads **The work item type import has completed** when the work item type is imported successfully. This will import the file (`/f`) `ProductBacklogItem.xml` into the Team Project Collection (`/c`) at the URL `http://localhost:8080/tfs`, which is in the Team Project (`/p`) demo.

When you create a new PBI (or open an existing one), you will see the **Product Code** field.



New Product Backlog Item with Product Code field

You can also see that after you have changed the state of a work item to **Approved**, you are not able to alter the **Product Code** field.

Adding a new portfolio to the process template

Lots of companies use different portfolios to categorize and define/understand the scope of work. The default **TFS Scrum template** contains a **Feature**, Product Backlog item portfolio. Today, we are going to create a level, **Epic**, which is above Feature. TFS supports up to seven different portfolio levels.

We aren't going to focus on what fields are available for the new **Epic Work Item Type**, but rather what is required to add it to our TFS process template.

Creating a new work item type definition

The first step is to create a copy of the existing `Feature.xml` in the `TypeDefinitions` folder (`C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\TypeDefinitions`) and name it `Epic.xml`. Open the `Epic.xml` file in your favorite editor. Then locate and change the name attribute on the `WORKITEMTYPE` node to `Epic`.

Adding a new portfolio backlog

Now, open the `ProcessConfiguration.xml` file that is located in `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\Process`. In the `PortfolioBacklogs` node, add the `.xml` shown as follows:

```
<PortfolioBacklog category="Microsoft.EpicCategory" pluralName="Epics"
singularName="Epic">
  <States>
    <State value="New" type="Proposed" />
    <State value="In Progress" type="InProgress" />
    <State value="Done" type="Complete" />
  </States>
  <Columns>
    <Column refname="System.WorkItemType" width="100" />
    <Column refname="System.Title" width="400" />
    <Column refname="System.State" width="100" />
    <Column refname="Microsoft.VSTS.Common.BusinessValue" width="50"
/>
```

```
    <Column refname="System.Tags" width="200" />
  </Columns>
  <AddPanel>
    <Fields>
      <Field refname="System.Title" />
    </Fields>
  </AddPanel>
</PortfolioBacklog>
```

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

This XML blurb is the same as the default Features, but with a small change on the first line. We are specifying that this is the `Microsoft.EpicCategory` category, and we changed the plural and singular names for the work item type to `Epics` and `Epic`. You will also need to add the attribute `parent="Microsoft.EpicCategory"` to the `Feature` portfolio to state that the new `Epic` category is the parent of `Feature`.

You will also need to find the `WorkItemColors` node and add the following node to specify the color to be used for the **Epic** work item type.

```
<WorkItemColor primary="FFFF5900" secondary="FFFFA200" name="Epic" />
```

This line simply specifies the primary and secondary colors to be used when displaying this work item on the boards and in grids. This helps users of TFS to easily identify a work item by its color, instead of looking for its work item type in the text.

Adding a new work item category

Next, open the `Categories.xml` file that is in `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking`, and add the following XML under the `cat:CATEGORIES` node:

```
<CATEGORY name="Epic Category" refname="Microsoft.EpicCategory">
  <DEFAULTWORKITEMTYPE name="Epic" />
</CATEGORY>
```

This will let TFS know that there is an `Epic` category available and the default work item type for the `Epic` category is our new `Epic` work item type.

Importing a new work item portfolio

Importing a new work item or changes made to a work item category will require you to import three files (the ones we just edited). You can accomplish this by opening the **Developer Command Prompt** for VS2013 and importing the files as discussed in the following sections.

Importing the work item type definition

Importing WITD is the same as what we did earlier. This is a regular import of the WITD using the `witadmin` tool.

```
cd C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem
Tracking\TypeDefinitions
witadmin importwitd /collection:http://localhost:8080/tfs /p:Demo
/f:Epic.xml
```

Your new `Epic` work item type now exists in TFS.

Importing the categories file

We can use the `importcategories` tool to import categories. This is shown as follows:

```
cd C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem
Tracking
witadmin importcategories /collection:http://localhost:8080/tfs /p:Demo
/f:Categories.xml
```

This process will update all category information in TFS.

Importing the process configuration file

We can use the `importprocessconfig` tool to import the process configuration files as follows:

```
cd C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem
Tracking\Process
witadmin importprocessconfig /collection:http://localhost:8080/tfs
/p:Demo /f:ProcessConfiguration.xml
```

This will import all the process template configuration files into TFS.

Summary

In this chapter, we covered adding new fields to an existing WITD, placing these fields in the form layout using the TFS Power Tools, and manually editing the `.xml` files. We also altered the workflow of an existing WITD by editing the `.xml` file. If you would like to find out more information on customizing the process template you can go to the following MSDN article `http://msdn.microsoft.com/en-us/library/ms243782.aspx`.

In the next chapter, we will be looking at how you can increase the Web Access usability by lighting up the dashboard of a team. We will also learn about writing a custom plugin to enhance the portfolio and task board.

www.allitebooks.com

2
Increasing Web Access Usability

In this chapter, we will cover the basic customization of the **TFS Web Access** interface as well as some extensibility points that can greatly increase the usability and productivity of users using Web Access. The topics covered in this chapter are as follows:

- Customizing the dashboard by adding tiles
- Changing the default columns for portfolio boards
- Adding the fields available in the portfolio add panels
- Creating a basic JavaScript plugin

This chapter will assume that you have read *Chapter 1*, *Customizing the TFS Process Templates for Your Business*, and therefore understand how to perform actions that were discussed in the chapter. The Process template used in this chapter is Microsoft Visual Studio Scrum 2013, and we will assume that you still have it saved in `C:\ProcessTemplates\` from the previous chapter.

Customizing TFS dashboard tiles

Having a customized dashboard in TFS is one of the biggest and easiest ways to show what is going on with a team's progress. Today, we will go over some of the things you can add to the dashboard of TFS.

Adding a tile to the dashboard

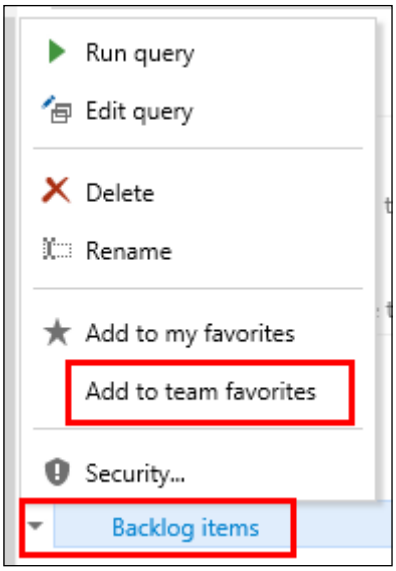
Surfacing query results as tiles on the dashboard can highlight very specific pieces of information within a team. In this section, we are going to add a query for the team's current backlog to the dashboard:

1. Open the TFS Web Access interface.
2. From the home page of a team, click on **View backlog**.
3. To create the backlog query, click on **Create query**, give the query the name `Backlog items`, and change the folder name to `Shared Queries`.
4. Click on **OK**.

Create a backlog query dialog

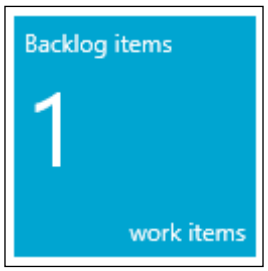
> The preceding steps will then create a query that will match what is shown on your team's backlog at the given point of time; if you alter the area paths or any other variable that will alter what displays on the team's backlog, you will need to update this query.

Above the backlog items grid, you will see a message that ends with the **Click here to view it** link; click this link to go to the query. You should see your query in the panel on the left under **Shared Queries**. Click on the arrow to the left of your query and then select **Add to team favorites**, as shown in the following screenshot:



Adding a query to team favorites

You will now see that the **Backlog items** query is shown in the queries panel under **Team Favorites**. Also, if you browse to the home page of the team, you will see a new tile that has the count of your backlog items, as shown in the following screenshot:



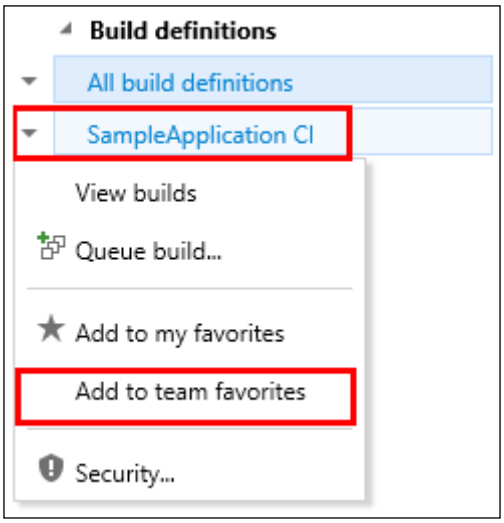
The work item query pinned to the team's dashboard

You can now click on this tile from your dashboard to quickly navigate to the query results. To remove a pinned query, click on the **View queries** link on the home page, then click on the query, followed by clicking on the arrow, and then from the menu select **Remove from team favorites**.

Pinning build definitions to the team dashboard

Most teams use **CI** (**continuous integration**) builds, and we are going to take any build that is currently set up in the TFS and simply show its status on the dashboard, next to our pinned query for **Backlog items**.

From the home page of a team, navigate to the build page using the **BUILD** link on the main navigation menu. Look for the build in the list of builds that you want to pin. Then, in the same way as we pinned the query, click on the arrow and then click on **Add to team favorites**, as shown in the following screenshot:

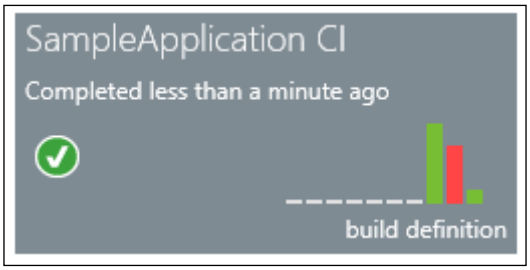


Adding a build to team favorites

The build will now show in the **TEAM FAVORITES** section, and also on the home page as a wide tile, as shown in the following screenshot:

Build pinned to the dashboard

You will see that this build failed. Finding out the reason for this failure is as easy as clicking on the build tile and then on the latest failed build, and this will show you the summary information of the build. You will have also seen that if your build breaks, a bug is created and assigned to the person who broke the build; this will now make the product backlog query result change from the previous chapter. Once you have fixed your build, you will see that the tile on the home page is updated, as shown in the following screenshot:



Build tile showing that the build has been fixed

To unpin this tile from the home page, navigate to **build definition** and click on the arrow and **Remove from team favorites**.

Pinning recent changes to the dashboard

Pinning source control folders to the dashboard is very useful for important or rarely changed parts of the code, such as framework or security. In this section, we are going to be pinning any changes that we have in TFS to the root folder of any source.

From the home page, navigate to the code section of Web Access using the **CODE** link in the main navigation pane. You will see that this page looks very similar to the query and build page, and the steps for pinning recent changes to the home page are therefore the same. Click on the arrow of any folder, file, or branch of code and click on **Add to team favorites**. The action performed in the previous line will show in the **team favorites** on the dashboard.

Recent changes pinned to the dashboard

You are able to click on this tile to navigate directly to the folder, file, or branch you pinned.

Changing the default columns for portfolio boards

Sometimes, it's useful to change the default columns of the portfolio boards. This is especially useful when you are customizing your process template and adding new fields that you wish to display on the portfolio backlog list. These changes will affect all users.

Open the `ProcessConfiguration.xml` file located at `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\Process`. Find the **Backlog items** and then the `PortfolioBacklog` node. You can see that there are six columns that show for this portfolio by default, and if you navigate to Web Access to the features portfolio backlog, these columns are currently displayed.

You are able to reorder these column nodes, or add new ones for any column you would like to display. For this sample, we are going to add the `Stream of Work` and `Product Code` fields that we added to the **Product Backlog Item** button in the previous chapter, and also remove the `System.WorkItemType` field from the displayed columns.

This is as simple as removing the following line from the column's child nodes:

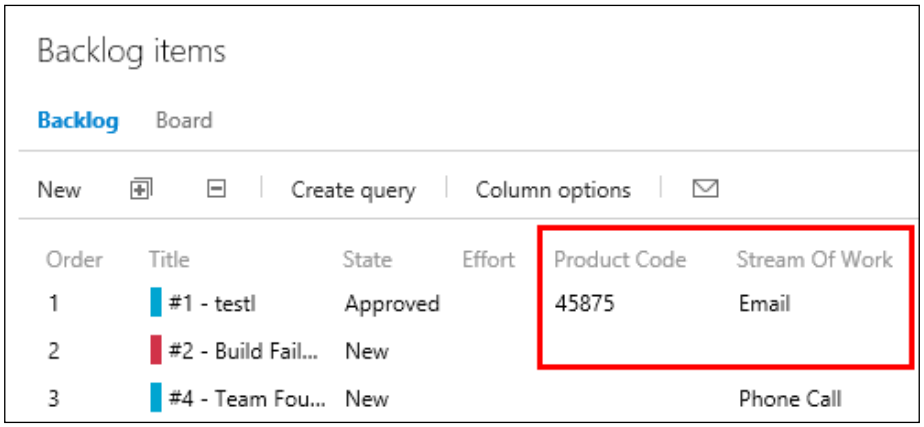
```
<Column refname="System.WorkItemType" width="100" />
```

And adding the nodes as shown in the following lines of code:

```
<Column refname="MyCompany.ProductCode" width="100" />
<Column refname="MyCompany.StreamOfWork" width="200" />
```

The last step in getting this change into TFS is importing the process configuration using the **witadmin importprocessconfig** tool.

Refresh the backlog items portfolio page and observe how the columns change, as shown in the following screenshot:



New columns for the backlog items portfolio

You can now see the columns for **Product Code** and **Stream Of Work** in the list and the Work item type column is no longer displayed.

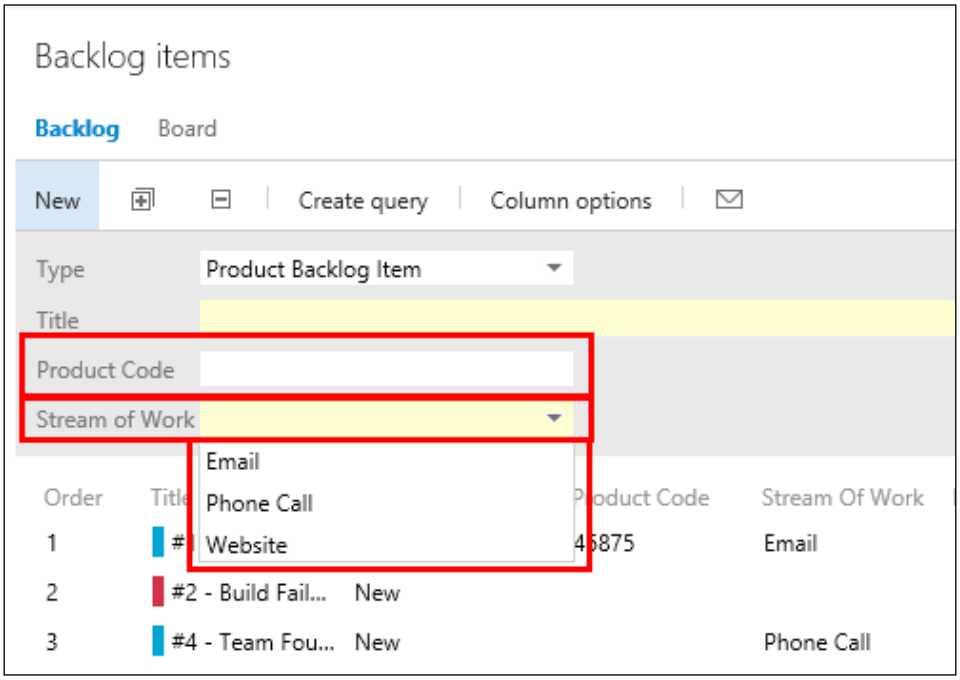
Adding fields to the add panels

When you are adding lots of work items to a backlog, it is convenient to use the quick add panel; however, what if you need to fill in fields that do not show in the quick add panel? This will make the process of adding work items slower and we are here to make our users more productive.

We are going to add the **Stream of Work** and **Product Code** fields to the quick add panel for the convenience of our users. Open the `ProcessConfiguration.xml` file located at `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\Process`. Now, find the **Backlog items** and then the `PortfolioBacklog` node. You will see that under the columns node that we worked with earlier, there is an **AddPanel** node. This node contains a **FIELDS** node that has a `FIELD` node for each of the fields that are to be shown in the quick add panel. Add the following code under the `FIELD` node for **Title**:

```
<Field refname="MyCompany.ProductCode" />
<Field refname="MyCompany.StreamOfWork" />
```

Again, to get this change into the TFS, use the witadmin importprocessconfig tool to import the process configuration. Refresh the backlog items portfolio to see the following changes:



New fields displaying in the add panel

You will now see that the **Product Code** and **Stream Of Work** fields both show in the add panel and have the same rules applied to them as when they are in a pop-up menu.

Creating a basic JavaScript plugin

Up until now in this chapter, we have been performing simple customizations that make it much easier for users to work in TFS. The next part that we are going to explore in Web Access is creating a basic JavaScript plugin that will display work item IDs on both the task and portfolio boards and the state of requirements on the task board.

> The JavaScript API for TFS is not an official extension point, meaning that it is not supported by Microsoft (this will not affect your TFS installation support) and does change between versions at which point your plugin could break. Normally, minor changes are required to your plugin to get it up and running. But still it is worth testing all plugins in a staging environment before applying them to a production environment where they may not work and then potentially break other functionality.

I prefer to use **TypeScript** as much as possible over raw JavaScript as it gives you many benefits that make it easier to prevent bugs (TFS 2012 was changed to TypeScript as well, which you can read up on Brian Harry's blog, *Typescript - a real world story of adoption in TFS*), but today we will create the plugin using only JavaScript.

Getting started with an empty plugin

To get started, create three files called `manifest.xml`, `MyCompany.WebAccess.Plugin.debug.js`, and `MyCompany.WebAccess.Plugin.min.js`. In the `manifest.xml` file, place the following XML:

```
<WebAccess version="12.0">
  <plugin name="MyCompany Plugin - Web Access" vendor="Gordon
Beeming" moreinfo="http://31og.com" version="1.0">
    <modules>
      <module namespace="MyCompany.WebAccess.Plugin"
loadAfter="TFS.Agile.TaskBoard.View"/>
      <module namespace="MyCompany.WebAccess.Plugin"
loadAfter="TFS.Agile.Boards.Controls"/>
    </modules>
  </plugin>
</WebAccess>
```

In the preceding code, once the `plugin` node has the attributes `name`, `vendor`, `moreinfo`, and `version`, we will be able to easily identify our plugin in the TFS Web Access admin area. Under the `modules` node, you will see that we have added two child `module` nodes. This informs TFS that we want to load our `MyCompany.WebAccess.Plugin` namespace after the `TFS.Agile.TaskBoard.View` and `TFS.Agile.Boards.Controls` namespaces, which are namespaces loaded on the task board and portfolio boards. You can get the base of this plugin from the sample code in the `MyCompany.WebAccess.Plugin - Base.js` file. If you have used the **RequireJs** module loader, you will notice that this syntax is very familiar.

www.allitebooks.com

In the base code, you will see a bit of code like the following:

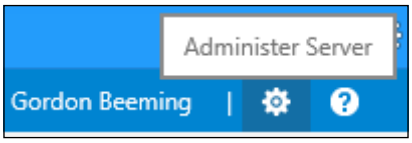
```
TfsWebAccessPlugin.prototype.initialize = function () {
    // place code here to get started
    alert('MyCompany.WebAccess.Plugin is running');
};
```

This `initialize` method is where you start gaining control of what is happening in Web Access. Take all the code in the base code and place it in the `debug.js` file.

Importing a plugin into TFS Web Access

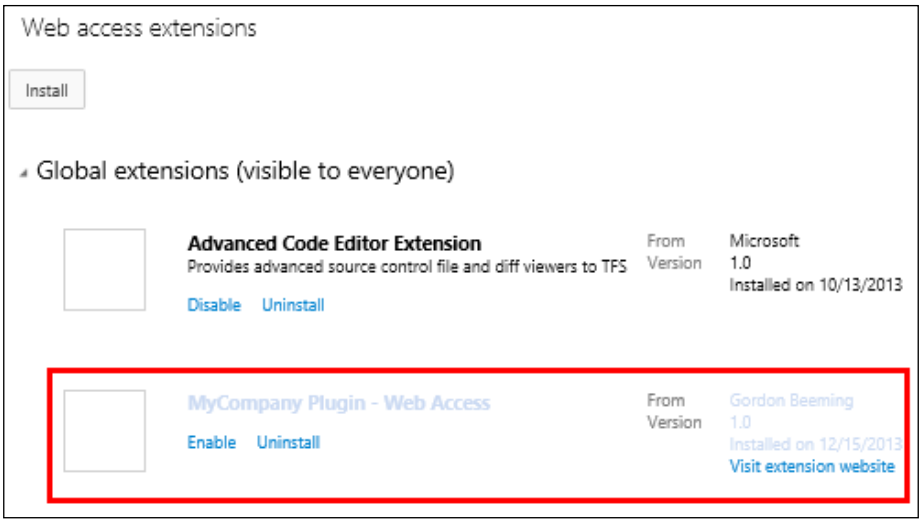
The first part of importing a plugin into TFS is to make sure that you have placed a minified version of your `*.debug.js` contents into your `*.min.js` file. Update the version of your plugin in the `manifest.xml` file, if required; for now, we will leave it at 1.0.

Zip the three files we created; the name of this ZIP file doesn't make a difference to the usage of the plugin. Browse to the server's home page and then click on the **Administer Server** button in the top-right corner as shown in the following screenshot:



The Administer Server Button

Click on the **Extensions** tab and then click on **Install**. In the model window, click on **browse** to browse for the ZIP file you created with the contents of the plugin and then click on **OK**. You will now see that the plugin is visible in the extensions screen but is currently not enabled. Click on **Enable** and then on **OK** to enable it, as shown in the following screenshot:



Web access extension when disabled

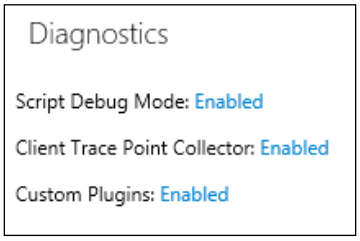
When you navigate to any of the boards, you will see the alert that we placed in the `initialize` function.

Setting up the debug mode

We have just imported our plugin into TFS, and this was quite a long process. Although it is fine if we upload our plugin into an environment, when we have finished creating our plugin, it becomes very time consuming when we need to make changes to the plugin. You have to go through this whole process to see the changes. So, we will use some tricks that will help us debug our extension.

Enabling the Script Debug Mode

Navigate to the TFS URL with `_diagnostics` appended at the end, that is, `http://gordon-pc:8080/tfs/_diagnostics`. On this page, we will click on the **Script Debug Mode** link, which should currently be disabled. This should also switch **Client Trace Point Collector** to **Enabled**, as shown in the following screenshot:



TFS diagnostics settings

This will now make TFS use the `debug.js` file instead of the `min.js` file. You will also see more requests for JavaScript files as each file is now streamed separately instead of being bundled together for better load performance. For this reason, it is probably very clear that this should not be enabled on a production environment.

Configuring a Fiddler AutoResponder rule

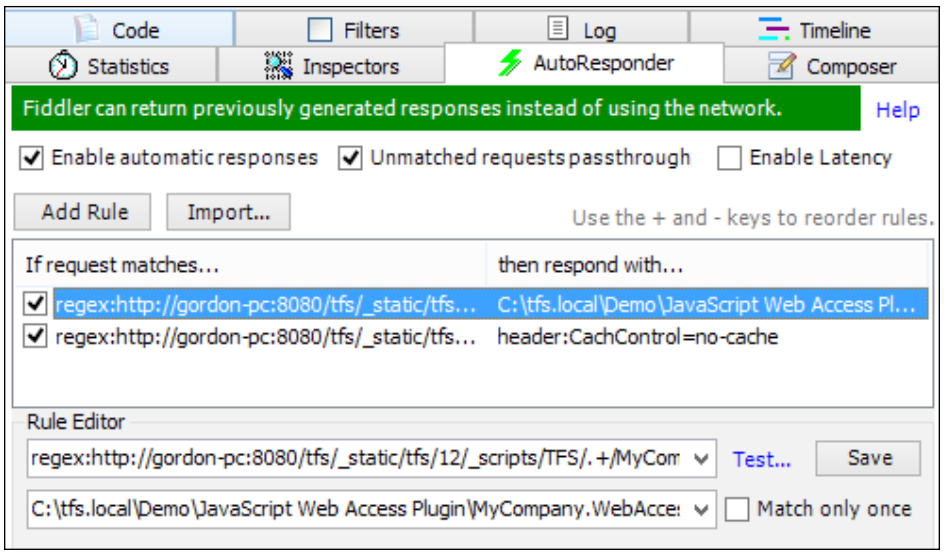
The next part is to configure Fiddler to automatically respond to any requests for your plugin from the server with your local `debug.js` file. This debugging tip was shared by Alexander Vanwynsberghe on his blog at the following link:

`http://www.alexandervanwynsberghe.be/debugging-tfs-web-access-customizations/`

You can download Fiddler from `http://fiddler2.com/`. We are going to use Fiddler to intercept the request for our plugins' JavaScript file from TFS and use our local version of the plugin.

The first step would be to start up Fiddler and make sure you can see the request for the `MyCompany.WebAccess.Plugin.js` file, which should have a URL similar to `http://gordon-pc:8080/tfs/_static/tfs/12/_scripts/TFS/debug//tfs/_plugins/1957/MyCompany.WebAccess.Plugin.js`.

In Fiddler, switch to the **AutoResponder** tab and check **Enable automatic responses** and **Unmatched requests passthrough**. Now click on **Add Rule** and in the **Rule editor** menu, use the `regex:http://gordon-pc:8080/tfs/_static/tfs/12/_scripts/TFS/.+/MyCompany.WebAccess.Plugin.js` rule; this will put a wildcard on the mode and plugin ID that is being used currently. In the second textbox, write down the full location of the `debug.js` file for this plugin and then click on **Save**. Add a second rule in the same pattern, but this time in the second textbox, use `header:CachControl=no-cache` and click on **Save**. You should see something similar to the following screenshot in Fiddler:



Fiddler AutoResponder rule added

This will now make Web Access use your local `debug.js` file for all requests for the plugin in TFS. To try this out, go to the `debug.js` file, change the alert to **we have added debugging**, and save the file. Refresh the board, and you will see that without any additional effort, the alert changed.

Adding information to display work items

The final code for this section can be found in the sample code but we will be going through some of the snippets that make a difference and are crucial to our plugin working correctly.

The easiest way to make use of these types of plugins is to change the HTML based on the information available in the HTML; this is useful for small changes, such as displaying the ID of work items on the work item cards on the boards. For this, you would, on initialization of your plugin, use the `setInterval` function in JavaScript and call the following function every 500 milliseconds:

```
function TaskBoardFunctions() {
    //replace IDs for tasks
    $("#taskboard-table .tbTile").each(function () {
        var id = $(this).attr("id");
        id = id.split('-')[1];
        $(this).find(".tbTileContent .witTitle").html("<span
            style='font-weight:bold;'>" + id + "</span> - " +
            $(this).find(".witTitle").html());
    });
    //replace IDs for tasks
    $("#taskboard-table .taskboard-row .taskboard-
        parent").each(function () {
        var id = $(this).attr("id");
        if (id != undefined) {
            id = id.split('_')[1];
            id = id.substring(1);
            $(this).find(".witTitle").html("<span style='font-
                weight:bold;'>" + id + "</span> - " +
                $(this).find(".witTitle").html());
        }
    });
}
```

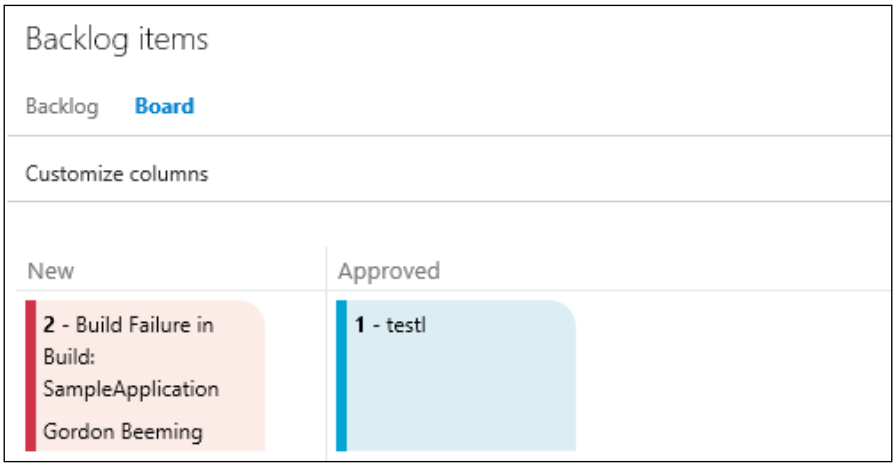
This function just looks for all work items on the page using the IDs that are specified in the attributes in the HTML elements to add the IDs to the UI.

A better way to do this would be to make use of the events in the API, and only make modifications to the displayed information when necessary. You would still use something similar to the preceding code for your initial loading to go through the board, and set all the information you would want to display; however, you would reply on the events to do any further updates. So, in this case, we would use the preceding code to scan for all the IDs on the page and then pass that through to a method, such as the following one, which will query the work item store. TFS has a configurable value that tells us the number of results that can be returned per query through the JavaScript API, and for this reason, we query 100 work items at a time; however, you can change this if it's not applicable to your plugin.

```
Core.prototype.loadWorkItemsWork = function (idsToFetch, onComplete,
that) {
    var takeAmount = 100;
    if (takeAmount >= idsToFetch.length) {
        takeAmount = idsToFetch.length;
    }
    if (takeAmount > 0) {
        that.WorkItemManager.store.beginPageWorkItems(idsToFetch.
splice(0,
    takeAmount), [
            "System.Id",
            "System.State"
        ], function (payload) {
            that.loadWorkItemsWork(idsToFetch, onComplete, that);
            $.each(payload.rows, function (index, row) {
                onComplete(index, row, that);
            });
        }, function (err) {
            that.loadWorkItemsWork(idsToFetch, onComplete, that);
            alert(err);
        });
    }
};
```

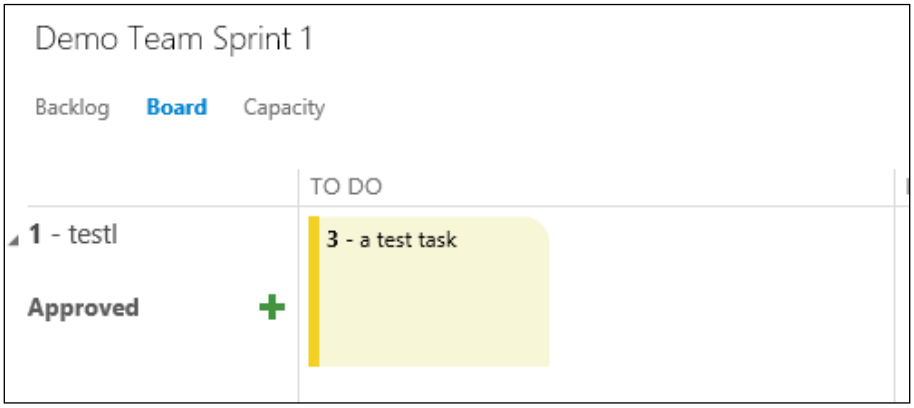
As you can see, we are querying the work item store for the ID and the state of each work item on the page. We are then passing this off to an `onComplete` function that is using jQuery to find the elements by ID. We then alter the displayed information to show the ID, and on the task board to show the state of the requirement.

If you use all the sample code and upload it into TFS, you will see a portfolio board like the one shown in the following screenshot:



IDs on the portfolio board

And on the task board, you will see the following screenshot:



IDs and State on task board

You can see that the tasks have IDs on them, which are the same as the portfolio boards, and the requirements listed on the left have IDs and their current states.

Summary

In this chapter, we covered customizing the TFS dashboard to display information that helps us find out a team's current status by pinning queries, build status, and recent changes to the source code. We then made some changes to the columns displayed in the portfolio backlog and the quick add panel. We finished off by going through what is required to create a TFS Web Access plugin.

I occasionally blog about tips and tricks for this topic on my blog at `http://31og.com/`.

In the next chapter, we will be looking at creating custom controls to be used within our work item definitions as well as displaying a web page inside our work item definitions.

3

Enhancing Work Items with Custom Controls

This chapter will cover creating custom controls for TFS. We will start with creating a Windows Forms control that will be displayed in the client applications, and then we'll create its web counterpart. We will then use this control in a work item definition. We will be covering the following topics:

- Creating a Windows Forms control for use in the client applications, including:
  - Writing the code
  - Deploying the control
  - Debugging the control
- Creating a control for use inside the Web Access, including:
  - Writing the code
  - Deploying and debugging the code
- Using the newly created control on an existing work item definition
- Displaying a web page with rich data inside a work item definition

There are some types of fields that would make you think, "If only I had a certain type of field I could...". Well, in this chapter, we are going to create a control that would come in very handy for most teams, and this control is a checkbox. You can make your custom controls as fancy as you would like; but remember that to fully implement it, you will need to create a Windows Forms version (for each version of Visual Studio being used) and a web version. Parts of this chapter will assume that you have been through the previous chapters and have already completed parts such as saving the Scrum process template to the filesystem.

www.allitebooks.com

Creating a Windows Forms control

First, we will create a Windows Forms control that will be used while displaying inside Visual Studio and other client applications.

Writing the code

The first step would be to create a new **class library** called `CustomTFSClientControls`. This class library can be a .NET 4.0 library (you need to create class libraries that are able to target the Work Item Tracking assembly for each version of Visual Studio you are supporting). Add a reference to `Microsoft.TeamFoundation.WorkItemTracking.Client.dll`, `Microsoft.TeamFoundation.WorkItemTracking.Controls.dll`, and `System.Windows.Forms.dll`. Rename the `Class1` file and class to `TfsCheckbox`. Now, we need to make our `TfsCheckbox` control inherit from `System.Windows.Forms.CheckBox` and `Microsoft.TeamFoundation.WorkItemTracking.Controls.IWorkItemControl`. Add the following static code to the class:

```
public TfsCheckbox()
{
    base.ThreeState = false;
    base.Checked = false;
    base.CheckedChanged += TfsCheckbox_CheckedChanged;
}
void TfsCheckbox_CheckedChanged(object sender, EventArgs e)
{
    FlushToDatasource();
}
public System.Collections.Specialized.StringDictionary Properties
{ get; set; }
public bool ReadOnly { get; set; }
public object WorkItemDatasource { get; set; }
public string WorkItemFieldName { get; set; }
public event EventHandler AfterUpdateDatasource;
public event EventHandler BeforeUpdateDatasource;
public void SetSite(IServiceProvider serviceProvider)
{
    serviceProvider.GetService(typeof(IWorkItemControlHost));
}
```

Now, we will actually start performing some logic for our control. We will need to add the `Clear` method, which will be called whenever TFS is required to clear out the value of our control. In our case, we will set the `Checked` element to `false`:

```
public void Clear()
{
    base.Checked = false;
}
```

The next step is to implement the `FlushToDatasource` method. This usually happens during the save operation. In most cases, data should be written into work items immediately on user change:

```
public void FlushToDatasource()
{
    if ((WorkItemDatasource as WorkItem) == null ||
(WorkItemDatasource as WorkItem).Fields[WorkItemFieldName] == null)
    {
        return;
    }
    (WorkItemDatasource as WorkItem).Fields[WorkItemFieldName].Value =
base.Checked ? "True" : "False";
}
```

The last method that we will need to implement is the `InvalidateDatasource` method; at this stage, the control should refresh its data from the work item and ignore any changes made to the state of the control. We are going to read the value of the current field and then change the checked state.

```
public void InvalidateDatasource()
{
    if ((WorkItemDatasource as WorkItem) == null ||
(WorkItemDatasource as WorkItem).Fields[WorkItemFieldName] == null ||
(WorkItemDatasource as WorkItem).Fields[WorkItemFieldName].Value ==
null)
    {
        return;
    }
    base.Checked = (WorkItemDatasource as WorkItem).
Fields[WorkItemFieldName].Value == "True";
}
```

We have now implemented all the code required for our Windows client control.

Next, we will need to add a `.wicc` file that will contain information to help the client applications find our control when they need to load it. Add an XML file called `TfsCheckbox.wicc` to your project, and make sure that the **Build Action** option is set to **Content**. Also, set the **Copy to Output Directory** property to **Copy if newer**. Populate this file with the following XML:

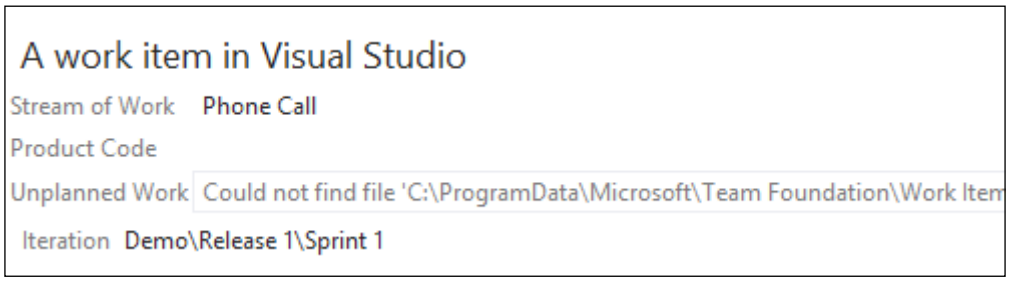
```
<?xml version="1.0"?>
<CustomControl xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Assembly>CustomTFSClientControls</Assembly>
  <FullClassName>CustomTFSClientControls.TfsCheckbox</FullClassName>
</CustomControl>
```

This definition contains the name of the assembly to look for (which has our control in it) and the full class name that will find our control in that assembly.

Deploying the code

The last part of creating a custom control for the client applications is deploying it to all users. To do this, build your custom control project, navigate to the output directory, and copy the `CustomTFSClientControls.dll` and `CustomTFSClientControls.wicc` files to `C:\ProgramData\Microsoft\Team Foundation\Work Item Tracking\Custom Controls\12.0`. This is the directory that the client applications will look for to find the configuration for the custom controls.

If, for some reason, you have deployed the control to an incorrect location or forgot to deploy it to a user, you will see the message **Could not find file 'C:\ProgramData\Microsoft\Team Foundation\Work Item Tracking\Custom Controls\12.0\TfsCheckbox.wicc'**.



Custom client control not found

As you can see, the label will still appear correctly and show the message in place of the custom control. Make sure that you close any client application after installing to ensure that they are able to detect the new custom control with issues.

Debugging the code

To debug our control, we will need to run Visual Studio as an administrator. Open the project that contains our custom control, open **project properties**, and then go to the **Build Events** tab. In the **Post-Build event command line** textbox, enter the following:

```
copy /Y "$(TargetDir)*.*" "C:\ProgramData\Microsoft\Team
Foundation\Work Item Tracking\Custom Controls\12.0\"
```

This will perform the deployment locally to your machine each time you build. To enable easy debugging, switch to the **Debug** tab, click on the **Start external program** radio button, and click on the **browse** button. Then, navigate to and select the `devenv.exe` file for Visual Studio (on x64, this is usually found in `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE`). Now, if you hit *F5* to start debugging, a new instance of Visual Studio will open. Then, if you open a work item that has your control on it, any breakpoints will be hit when that functionality is used—as you would expect with regular debugging.

Creating a control for use inside the Web Access

Now that we have built and deployed a client custom control, we will need to create a web control that will be used for displaying inside the Web Access.

Writing the code

Developing the web version of our control will be similar to our JavaScript plugin that we created in the previous chapter. To get started, create the `manifest.xml` file and the two JavaScript files: `CustomTFSClientControls.TfsCheckbox.debug.js` and `CustomTFSClientControls.TfsCheckbox.min.js`. Note that our JavaScript file names need to be the same as the namespace and class name; so basically, it's a match to the `FullClassName` class in our `.wicc` file in the client application. In our `manifest.xml` file, add the following XML:

```
<WebAccess version="12.0">
  <plugin name="MyCompany Controls - Web Access" vendor="Gordon
      Beeming" moreinfo="http://31og.com" version="1.0">
    <modules>
      <module namespace="CustomTFSClientControls.TfsCheckbox"
          kind="TFS.WorkItem.CustomControl"/>
    </modules>
  </plugin>
</WebAccess>
```

As you can see, this is almost the same as before. The only real difference here is that we add `kind="TFS.WorkItem.CustomControl"` instead of specifying the `loadAfter` attribute. This informs TFS that this `CustomTFSClientControls.TfsCheckbox` module is meant for a TFS custom control. In our `*.debug.cs` file, start with the following code:

```
TFS.module("CustomTFSClientControls.TfsCheckbox",
    [
        "TFS.WorkItemTracking.Controls",
        "TFS.WorkItemTracking",
        "TFS.Core"
    ], function () {
        // Replace this comment with the next steps code
        witControls.registerWorkItemControl("TfsCheckbox",
            TfsCheckbox);
        return { TfsCheckbox: TfsCheckbox };
    });
```

The preceding code covers the base of our custom control module. It specifies the references that we will require when our control runs. It also registers our work item control for TFS, which we can use when required. Next, we will need to define some of the options required for our custom control module. Add the local variables, constructor inherit function, and base `inherit` function with the following code:

```
var witOM = TFS.WorkItemTracking;
var witControls = TFS.WorkItemTracking.Controls;
var delegate = TFS.Core.delegate;
function TfsCheckbox(container, options, workItemType) {
    this.baseConstructor.call(this, container, options,
        workItemType);
}
TfsCheckbox.inherit(witControls.WorkItemControl, {
    // Replace this comment with the next steps code
});
```

The next step is to specify the options for the `inherit` function. We'll set `_control` to `null` and define the `clear` callback:

```
_control: null,
clear: function () {
    this._control.checked = false;
},
```

As you can see, we are setting the checked state to `false` on clear in the same way as we did for the client custom control. Next, we will define the `invalidate` callback:

```
invalidate: function (flushing) {
    if (this._getField().getValue() == "True"){
        this._control.attr("checked","checked");
    }
    else{
        this._control.removeAttr("checked");
    }
},
```

Again, this follows the same logic as the client custom control. The last part of this code is to define the `_init` callback:

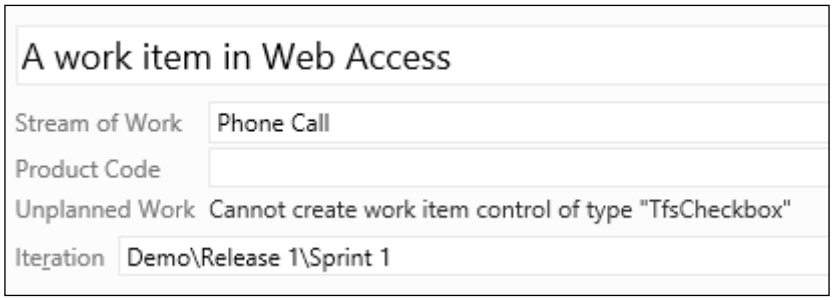
```
_init: function () {
    this._base();
    this._control = $("<input
        type='checkbox'/>").appendTo(this._container);
    var that = this;
    this._control.bind('change', function () {
        if ($(this).is(':checked')) {
            that._getField().setValue("True");
        } else {
            that._getField().setValue("False");
        }
    });
}
```

In our `_init` callback, we are setting our `_control` variable to be a checkbox and appending it to the container that is provided by TFS for our control to work within. We then **bind** the `change` event of our checkbox, so that we can change the value of the field when the checkbox value is changed.

Deploying and debugging the code

To deploy our custom web control, we will follow the same steps that are used to deploy our JavaScript plugin (refer to the *Importing a plugin into TFS Web Access* section in *Chapter 2*, *Increasing Web Access Usability*). The debugging techniques are also the same as what we used for the JavaScript plugin.

If, for some reason, you have not deployed the control, you will see the message **Cannot create work item control of type "TfsCheckbox"**.



Custom web control not found

As you can see, the label will still appear correctly and show the message in place of the custom control.

Using the control on an existing work item definition

To use our new control in a WITD, we need to go back to the process template, add a new field, and then set the control to be our control.

Start off by opening the `ProductBacklogItem.xml` WITD, which you should be able to locate in the path `C:\ProcessTemplates\Microsoft Visual Studio Scrum 2013\WorkItem Tracking\TypeDefinitions`. Next, add a new field called `Unplanned Work` with a reference name `MyCompany.UnplannedWork` and the type as `String`. For the rules of this field, add a `DEFAULT` value of `False` and allowed values of `True` and `False`. Your field definition should look like this:

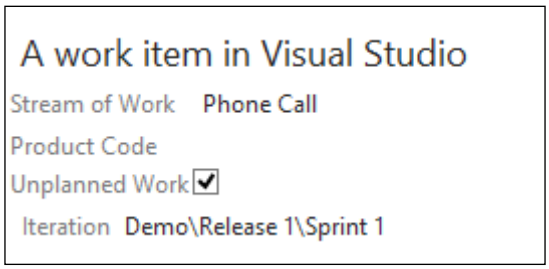
```
<FIELD name="Unplanned Worked" refname="MyCompany.UnplannedWork"
    type="String">
<DEFAULT from="value" value="False" />
<ALLOWEDVALUES expanditems="true">
  <LISTITEM value="True" />
  <LISTITEM value="False" />
</ALLOWEDVALUES>
</FIELD>
```

Next, add your control to your layout. To do this, add a new control, set the **Label** of the control to `Unplanned Work`, and the **Field Name** to `MyCompany.UnplannedWork`. This time, unlike in the previous chapters, we will be setting the **Type** as well; it will be the name of our control: **TfsCheckbox**. Make sure that your control looks like the following XML:

```
<Control FieldName="MyCompany.UnplannedWork" Type="TfsCheckbox"
Label="Unplanned Work" LabelPosition="Left" />
```

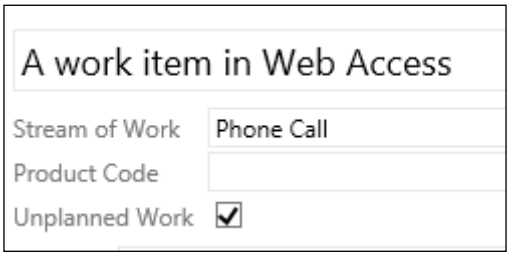
Import the product backlog item WITD using the **witadmin importwitd** tool.

If you open a product backlog item in a client application and Web Access, you can see that our control works as expected. When we open a work item in an application such as Visual Studio or Test Manager, we will see a message similar to the following one:



Custom client work item control loaded

And when it is opened in the Web Access, we will see the web version:



Custom web work item control loaded

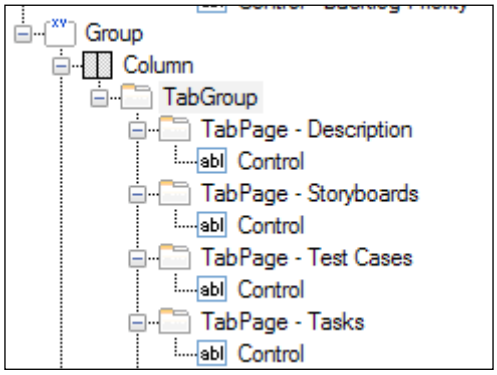
As we can see, there is no real difference in how these controls are displayed and how we are able and required to control all the logic that is performed by our controls.

Displaying a web page with rich data inside a WITD

There's a strong possibility that you will set up reporting for TFS; but for small pieces of data, such as finding out what Test Plans a Test Case is a part of, you wouldn't want your users to have to leave TFS, go into another system, and then return to TFS again. The solution for displaying this type of data is to embed it in the work item definition, so that when a user selects a work item they are able to easily gain access to this information.

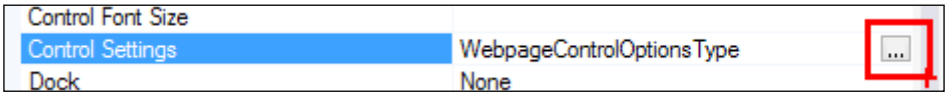
We aren't going to create a website for this demo, but we will set up a web page control that will load a URL and pass some variables from the loaded work item as part of the query string.

We will start off by opening the Product Backlog Item WITD with the TFS Power Tools using the Process Template Editor, going to the **Layout** tab, and locating the **TabGroup** folder to the left of the form, as shown in the following screenshot:



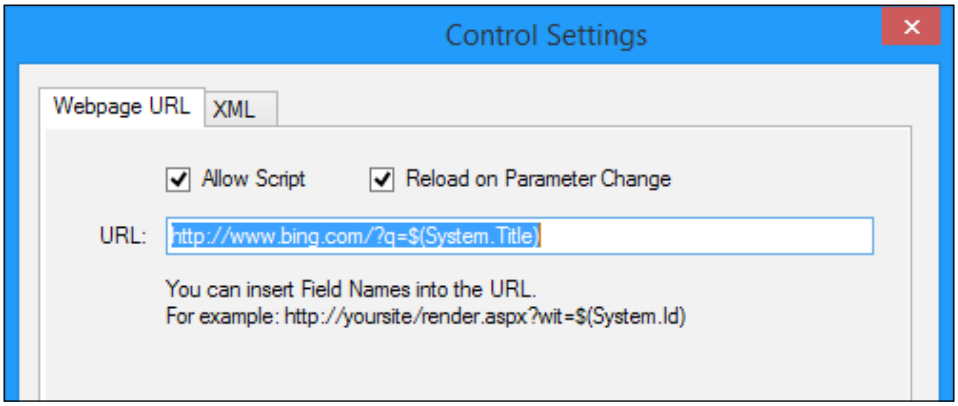
The TabGroup to the left of the Product Backlog Item WITD

Right-click on **TabGroup** and click on **New Tab Page**. Then, set the **Label** on the new tab to `Web Page Demo`. Right-click on the tab group, click on **New Control**, and then empty the label for the control. At the bottom of the list of properties, you will see the **Type** property; set its value to `WebpageControl`. Notice that as you do this, the **Control Settings** property values changes to **WebpageControlOptionsType**. Click on the button on the right to open the settings window:



Layout Control Settings button

When the settings pop up, make sure that you check the **Allow Script** and **Reload on Parameter Change** checkboxes. This will simply allow the web page we load to run scripts and refresh anytime any of the parameters that are linked to the webpage URL are changed. Also, set the URL to `http://www.bing.com/?q=$(System.Title)` as shown in the following screenshot:



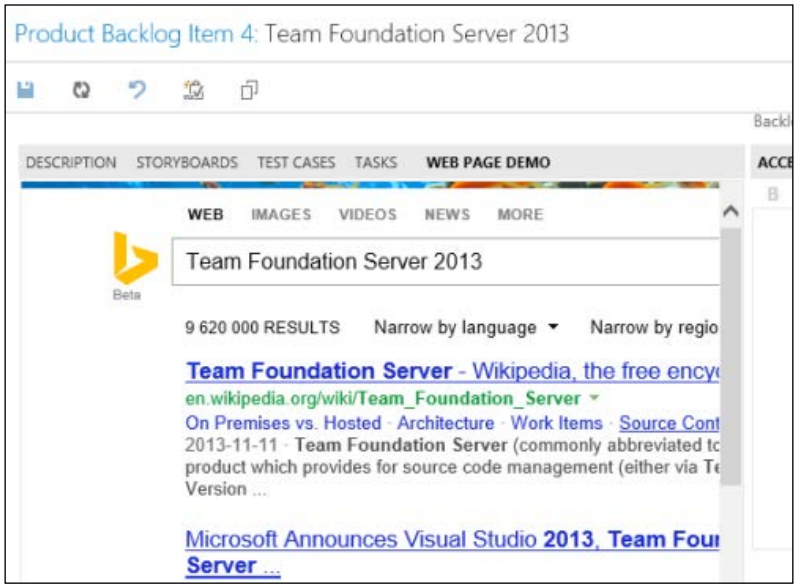
Control settings

Notice how we used a placeholder here to automatically set the search query string parameter to the title of our work item. All these changes made using the TFS Power Tools resolves to the XML shown as follows:

```
<Tab Label="Web Page Demo">
<Control Type="WebpageControl" LabelPosition="Left">
  <WebpageControlOptions AllowScript="true"
      ReloadOnParamChange="true">
    <Link UrlRoot="http://www.bing.com" UrlPath="?q={0}">
      <Param Index="0" Value="System.Title" />
    </Link>
  </WebpageControlOptions>
</Control>
</Tab>
```


www.allitebooks.com

To try this control out, import the Product Backlog Item WITD using the witadmin importwitd tool. This control will work in both client apps and the Web Access. Let's open a work item and check what we can see:



WebPageControl loaded with a search from the work item title

Notice how the web page is updated with new search results as the value of the title changes.

For a good example on how this can be used, take a look at a post by Mathias Olausson on the Microsoft MVP blog at the following link:

`http://msmvps.com/blogs/molausson/archive/2013/07/28/extending-tfs-work-item-using-a-web-page-and-the-webpagecontrol.aspx`

Summary

In this chapter, we stepped through the creation of custom controls that will be used within TFS for both client applications and the Web Access. We covered how to deploy the custom controls and what messages would show up if we didn't deploy them correctly. Then, we used one of the built-in controls that has endless possibilities and uses.

You can also look online at `http://witcustomcontrols.codeplex.com` for other examples of custom work item controls that have been created by the community.

In the next chapter, we will be looking into the creation of server plugins that are able to perform actions as changes are made to data in TFS.

4
Creating Server Plugins

This chapter will cover creating TFS server plugins. We will have a look at two types of server plugins, the examples of which will illustrate both decision points and notifications. Topics that we will be covering in this chapter include the following:

- Why use a TFS plugin over a check-in policy?
- The basics of a server plugin, including:
  - Setting up the code
  - Deploying server plugins
  - Debugging server plugins
- Creating a check-in plugin to force a comment on the framework code check-ins
- Creating a work item changed plugin that will make sure the ID of a work item prefixes the work item title

Server plugins can be useful when you want to do trigger-type changes; for example, when a check-in is made to a certain type of file or when details in a work item match a certain criteria.

Using a TFS plugin over a check-in policy

One of the reasons why you might want to use check-in policies is the fact that you are able to override check-in policies, whereas with server plugins, you aren't able to override them because they run on the server and not on the client. Because server plugins run on the server and not on the client, maintaining them is a lot easier as you can configure them in a centralized place that will cover all your team projects, instead of a per-project configuration as with the check-in policies. Server plugins are also much easier and quicker to deploy as it's one place on the server that places binaries instead of installing check-in policies per TFS client.

The basics of a server plugin

This section will take you through some of the basics that are required for creating server plugins for TFS.

Setting up the code

Setting up the code for a server plugin is really easy. Create a .NET 4.0 class library project and add references to `Microsoft.TeamFoundation.Framework.Server.dll`, which can be found in the `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\Web Services\bin` location on the TFS server; `Microsoft.TeamFoundation.Common.dll`, which is located at `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0`; and `Microsoft.VisualStudio.Services.WebApi.dll`, which is located at `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v4.5`. Next, add the `Microsoft.TeamFoundation.Framework.Server.ISubscriber` interface to your subscriber class and implement the interface. Set the name and priority of your plugin as shown in the following code:

```
public string Name
{
    get { return "MyCompany.ServerPlugin"; }
}
public SubscriberPriority Priority
{
    get { return SubscriberPriority.Normal; }
}
```

The name will be used to identify your plugin amongst other plugins, and the priority will determine the part when your plugin runs from all the server plugins.

We will get all the events for all team projects in the collection. This can be both a performance and security concern.

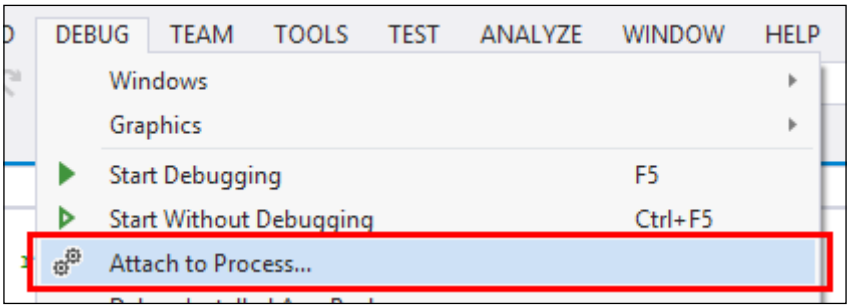
The other two methods that are part of the interface would contain code that is specific to the type of the plugins that you are creating.

Deploying server plugins

To deploy your server plugin, first compile an assembly that contains one or more server plugins, and copy that assembly and any dependent assemblies into `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\Web Services\bin\Plugins` on the TFS application tiers. Please be aware that the TFS site will be refreshed with the changing of these files, and users could experience performance issues and possible loss of data when the site is started again.

Debugging server plugins

Because TFS runs in **IIS** (**Internet Information Services**), debugging a server plugin is the same as using a regular web application. Make sure Visual Studio is running in the administrator mode if TFS is on the same machine as Visual Studio. Next, go to **DEBUG | Attach to Process...** on the menu, as shown in the following screenshot:

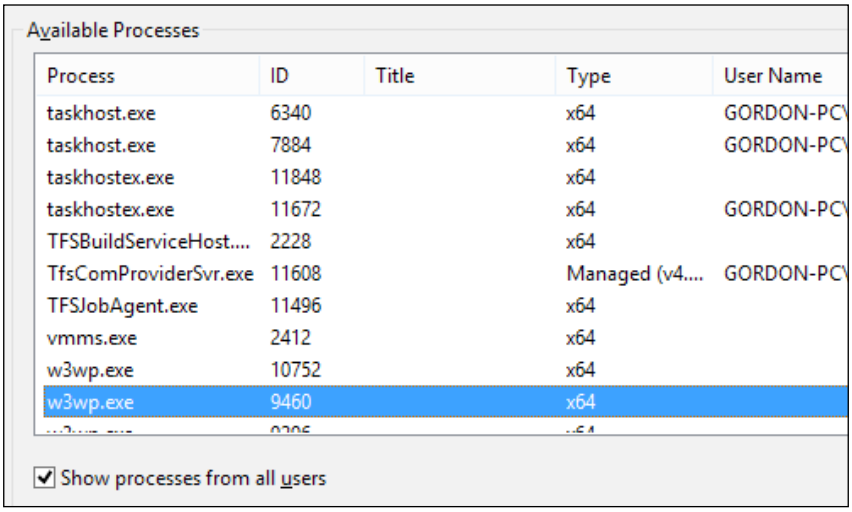


Attaching to process

After that, you need to locate the `w3wp.exe` process in the task manager, which is running TFS. There will normally be more than one process with the name `w3wp.exe` in the list, and you must make sure that you select **Show processes from all users** to show the `w3wp.exe` process in the list. If TFS is on a different server, you will need to use remote debugging, which you can read more about at the following link:

`http://msdn.microsoft.com/en-us/library/y7f5zaaa.aspx`

The example references to the store applications but the steps are the same. Once you have installed the remote debugging tool, you will be connected to the remote machine. Then, select `w3wp.exe` on that machine, as shown in the following screenshot:



The Available Processes window

If you are attached to the correct `w3wp.exe` process, you will be able to perform an action in TFS that would trigger your plugin, and you will be able to debug your code as you would normally do.

Creating a check-in plugin

In TFS, you are able to switch on a check-in policy that specifies that the users must specify a comment. The only problem with this is that you are able to override the policy and specify a reason why you don't want to provide a comment. Now, we are going to create a server plugin that will require a comment to be provided if any code is checked in to any folder or subfolder of a folder called `Framework`. Note that because this is a server plugin, the users will check in as usual and then get a message specifying that they need to add a comment.

To get started, create a new class named `FrameworkCommentRequirementSubscriber` by following the setup instructions from earlier. Name it `MyCompany.FrameworkCommentRequirementSubscriber`, and then set the priority to `SubscriberPriority.High`. Next, also add a reference to `Microsoft.TeamFoundation.VersionControl.Server.dll`, which is located at `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\Web Services\bin`. In the `SubscribedTypes` method, change the code with the following one:

```
public Type[] SubscribedTypes()
{
    return new Type[] { typeof(CheckinNotification) };
}
```

Also, add a `using` reference like the following one:

```
using System.Diagnostics;
using Microsoft.TeamFoundation.Framework.Server;
using Microsoft.TeamFoundation.VersionControl.Server;
```

This informs TFS that you want to subscribe to events that are of the `CheckinNotification` type. The `ProcessEvent` has three `out` parameters, so the first thing we will do is add the following code to set them as our default responses so that everything goes as planned:

```
statusCode = 0;
properties = null;
statusMessage = string.Empty;
```

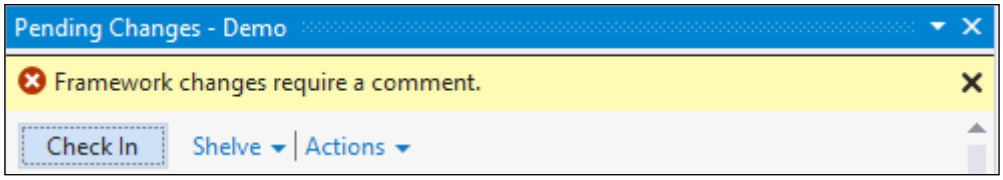
We will also add the following code for handling the default return value and also to check whether there is an error in the plugin or not:

```
try
{

    return EventNotificationStatus.ActionPermitted;
}
catch(Exception ex)
{
    // log the error and fail the check in
    statusMessage = "Error in plugin '" + Name + "', error details: "
+ ex.ToString();
    EventLog.WriteEntry("TFS Service", statusMessage,
EventLogEntryType.Error);
    return EventNotificationStatus.ActionDenied;
}
```

Now for the core logic for our plugin, we will first want to check if the current notification type is a **decision point**; we will check if a comment was provided for the request, and finally if there was no comment, we will check all the submitted items to see if any of them are in the `Framework` folder. If we find any items in a `Framework` folder, we will set the status message and return `ActionDenied`. To do all of this, add the following code inside the `try` block above the default `return EventNotificationStatus.ActionPermitted`:

```
if (notificationType == NotificationType.DecisionPoint &&
    notificationEventArgs is CheckinNotification)
{
    CheckinNotification args = notificationEventArgs as
CheckinNotification;
    // Is there a comment
    if (string.IsNullOrEmpty(args.Comment))
    {
        // Check each item checled in
        foreach (string item in args.GetSubmittedItems(requestContext
))
        {
            // if the server path of the item
            // contains /framework/ fail the checkin
            if (item.ToLower().Contains("/framework/"))
            {
                statusMessage = "Framework changes require a
comment.";
                return EventNotificationStatus.ActionDenied;
            }
        }
    }
}
```

Now that we have all the code set up for our plugin, you can build and deploy the assembly as described in the *Deploying server plugins* section of this chapter. If you check in the code, and not in the `Framework` folder, you will see that it works normally; but as soon as you check in the code, that is, in the `Framework` folder without a comment, you will get an error similar to what is shown in the following screenshot:



Framework code check error

As you can see, our plugin is working correctly. From here, you could extend this plugin to specific teams to look up folders to block, if you wanted, as this plugin currently applies the "Framework comment required" logic to every check-in for all team projects in the TFS instance where it is installed.

Creating a work item changed plugin

Work item changed plugins are great for creating triggers on work items that meet a certain criteria. In this section, we are going to use a work item changed server plugin to change the title of any work item that has a title that doesn't start with `#WorkItemID - `.

To get started, carry out the following steps:

1. Create a new class named `IdInTitleRequirementSubscriber` by following the setup instructions from earlier. Name it `MyCompany.IdInTitleRequirementSubscriber` and then set the priority to `SubscriberPriority.Normal`.
2. Next, add a reference to `Microsoft.TeamFoundation.WorkItemTracking.Server.Dataaccesslayer.dll`, which is located at `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\Web Services\bin`, and `Microsoft.TeamFoundation.Client.dll` and `Microsoft.TeamFoundation.WorkItemTracking.Client.dll`, which can both be found at `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0\`. In the `SubscribedTypes` method, change the code with the following snippet:

   ```
   public Type[] SubscribedTypes()
   {
       return new Type[] { typeof(WorkItemChangedEvent) };
   }
   ```

3. Add a `using` reference like the following:

   ```
   using System.Diagnostics;
   using Microsoft.TeamFoundation.Client;
   using Microsoft.TeamFoundation.WorkItemTracking.Server;
   using Microsoft.TeamFoundation.WorkItemTracking.Client;
   using WorkItem =
   Microsoft.TeamFoundation.WorkItemTracking.Client.WorkItem;
   ```


www.allitebooks.com

This informs TFS that you want to subscribe to events that are of the `WorkItem ChangedEvent` type. The base implementation for the `ProcessEvent` method is the same as before and is described in the following code:

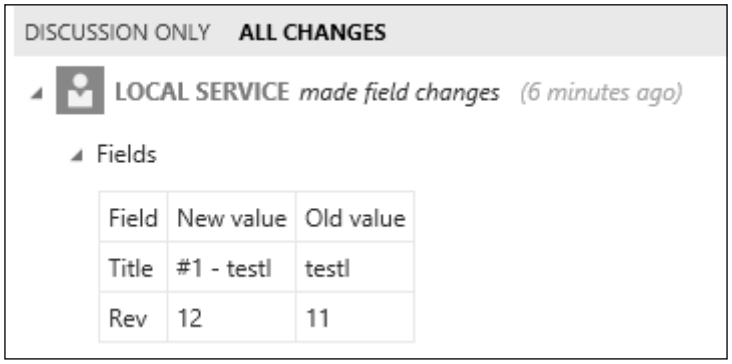
```
statusCode = 0;
properties = null;
statusMessage = string.Empty;

try
{

    return EventNotificationStatus.ActionPermitted;
}
catch (Exception ex)
{
    statusMessage = "Error in plugin '" + Name + "', error details: "
+ ex.ToString();
    EventLog.WriteEntry("TFS Service", statusMessage,
EventLogEntryType.Error);
    return EventNotificationStatus.ActionDenied;
}
```

For our implementation inside the `try` block, we will be checking if our notification type is set to `Notification`, and we will then check the work item title to see if it matches `#WorkItemNumber`, as we want it to. And if it doesn't, we will load the work item out of TFS and make the title what we want it to be. To do this, you should use the following code:

```
if (notificationType == NotificationType.Notification &&
    notificationEventArgs is WorkItemChangedEvent)
{
    WorkItemChangedEvent args = notificationEventArgs as
WorkItemChangedEvent;
    if (!args.WorkItemTitle.StartsWith("#" + args.CoreFields.
IntegerFields.First(o => o.Name == "ID").NewValue + " - "))
    {
        TfsTeamProjectCollection tfsServer = new TfsTeamProjectCollect
ion(GetTFSUri(requestContext));
        WorkItemStore workItemStore = tfsServer.
GetService<WorkItemStore>();
        WorkItem workItem = workItemStore.GetWorkItem(args.CoreFields.
IntegerFields.First(o => o.Name == "ID").NewValue);
        workItem.Title = "#" + workItem.Id + " - " + workItem.Title;
        workItem.Save();
    }
}
```

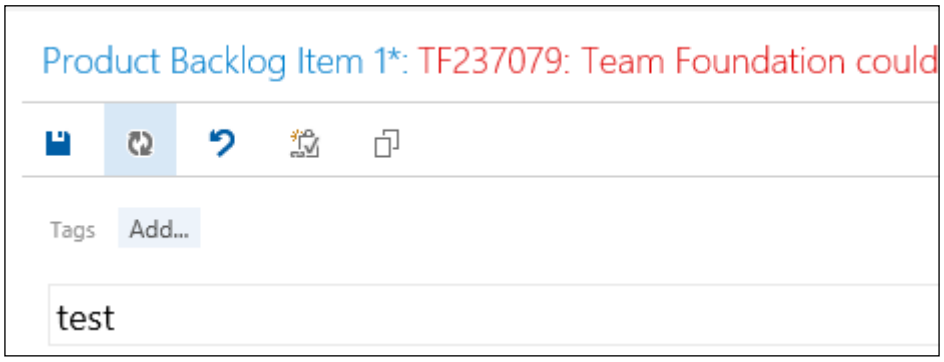
Deploy this server plugin the same way you did earlier, and you will now see that when you save a work item and refresh it a couple of seconds later, it gets updated to include the `#ID` in the title.

When the `Save()` method is called in the preceding code, it is a regular save of the work item; so, a new revision is created with the user set to the account that the TFS Web Access is running:



Work item changes made by LOCAL SERVICE

Be very careful while making changes to work items in server plugins, like we just did, because if any user wants to make changes to this work item that conflicts with what is happening in the server plugin, they will get the **TF237079: Team Foundation could not update the work item because it was already updated by another user, it does not exist, or you do not have permission to update it** error. This is shown in the following screenshot:



The error obtained when server plugin credentials conflict with user changes

Of course, this error will also happen if two users make a change to the same field without refreshing to get the other's changes first. But with a plugin like the one we just created it will happen more often. To demonstrate this, set the title of a work item to `test`, count to five, and then change the title to `testing` and click on **Save**.

Summary

In this chapter, we went through creating TFS server plugins, we went through how to deploy them, and what you need to do to debug them.

In the next chapter, we will be looking at creating TFS scheduled jobs. We will also be using a scheduled job to replace the work item changed server plugin created in this chapter, and discuss why that would be a better place for any logic that changes the work item's data.

5
Creating TFS Scheduled Jobs

In this chapter, we will cover the creation of jobs in TFS and demonstrate the updating of work items on a schedule. The topics covered in this chapter are:

- Creating a job that will be able to perform the same job as the work item changed plugin, as discussed in the previous chapter, which includes:
    - Writing the code
    - Deploying the code
    - Registering the custom job and adding it to a schedule
    - Verifying that the job is installed
- Checking the job history
- Debugging a TFS job
- Deregistering the custom job with TFS

In TFS, jobs are the best place to place logic that you would want to run on a schedule, unlike the server plugins that run on every work item change and only once per change. It is important to know though that the **TFS Background Job Agent** is not designed for time-critical jobs such as seconds, minutes, and so on.

Creating a TFS job

Today, we will be creating a TFS job that can replace our work item's changed server plugin that we created in the previous chapter. One of the reasons we would want to do this is that we are able to only run our logic when the last-changed time of the work item is after a couple of minutes. This will give users the time to complete what they are doing with the work items. Also, try to avoid getting the **TF237079 error** that was shown in the previous chapter.

Writing the code

To get started, create a new .NET 4.5 class library called `TFS.ScheduledJobs` and add a reference to the `Microsoft.TeamFoundation.Framework.Server.dll` file. This file is located in the directory `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\Web Services\bin` and `Microsoft.TeamFoundation.Common.dll`, which is in `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0` and `Microsoft.VisualStudio.Services.WebApi.dll`. This is located in `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v4.5`. Create a new class called `IdInTitleRequirementJob` and make it inherit from `ITeamFoundationJobExtension`. Then implement the interface. This should now give you the following code:

```
using System;
using Microsoft.TeamFoundation.Framework.Server;

namespace TFS.ScheduledJobs
{
    public class IdInTitleRequirementJob : ITeamFoundationJobExtension
    {
        public TeamFoundationJobExecutionResult Run(TeamFoundationRequ
estContext requestContext,
                TeamFoundationJobDefinition jobDefinition,
                DateTime queueTime,
                out string resultMessage)
        {
            throw new NotImplementedException();
        }
    }
}
```

Now, add a reference to `Microsoft.TeamFoundation.Client.dll` and `Microsoft.TeamFoundation.WorkItemTracking.Client.dll`, which can be found in `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0\`. The logic that we are going to use will be very basic. We will get a full list of all work items and then do the same validation as we did in the previous chapter for the title. After implementing the logic, the `Run` method should look like the following code:

```
try
{
    TfsTeamProjectCollection tfsServer = new TfsTeamProjectCollection(
GetTFSUri(requestContext));
    WorkItemStore workItemStore = tfsServer.
GetService<WorkItemStore>();
    List<WorkItem> changedWorkItems = new List<WorkItem>();
    foreach (WorkItem workItem in workItemStore.Query("SELECT * FROM
WorkItems"))
    {
        if (!workItem.Title.StartsWith("#" + workItem.Id + " - "))
        {
            workItem.Title = "#" + workItem.Id + " - " + workItem.
Title;
            changedWorkItems.Add(workItem);
        }
    }
    if (changedWorkItems.Count > 0)
    {
        workItemStore.BatchSave(changedWorkItems.ToArray());
        resultMessage = changedWorkItems.Count + " work item titles
updated.";
    }
    else
    {
        resultMessage = "no work item titles to update.";
    }
    return TeamFoundationJobExecutionResult.Succeeded;
}
catch (Exception ex)
{
    resultMessage = "Job Failed: " + ex.ToString();
    EventLog.WriteEntry("TFS Service", resultMessage,
EventLogEntryType.Error);
    return TeamFoundationJobExecutionResult.Failed;
}
```

As you can see, we are queuing all changes and doing `BatchSave` at the end instead of saving each work item that needs to be updated. We are also setting the `resultMessage` variable with the amount of items that we update, so that we can view it in the job history at a later point.

Deploying the code

Deploying a TFS job is very simple. All we need to do is copy the `TFS.ScheduledJobs.dll` and `TFS.ScheduledJobs.pdb` files into the plugins folder for the TFS Job Agent, which is `C:\Program Files\Microsoft Team Foundation Server 12.0\Application Tier\TFSJobAgent\Plugins`.

Registering the custom job and adding it to a schedule

To register our TFS job, we will create a new console application and add references to `Microsoft.TeamFoundation.Client.dll` and `Microsoft.TeamFoundation.Common.dll`, which can be found in `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0\`. To register the job, we need to create a job definition that will consist of the friendly name of our job and the full name of the class including the namespace. We will then set `EnabledState` of our job definition to `Enabled`, and create a schedule. We can create a schedule by passing the start date, the start time, and the interval in seconds required for our job to run. Finally, we will need to call the job service to save this job definition. To do all of this, add the following code to the `Main` method in the `Program.cs` file.

```
var tfsUri = new Uri("http://localhost:8080/tfs");
var config = TfsConfigurationServerFactory.
GetConfigurationServer(tfsUri);
var jobService = config.GetService<ITeamFoundationJobService>();
var jobDefinition = new TeamFoundationJobDefinition("ID in Title
Requirement Job", "TFS.ScheduledJobs.IdInTitleRequirementJob");
jobDefinition.EnabledState = TeamFoundationJobEnabledState.Enabled;
jobDefinition.Schedule.Add(new TeamFoundationJobSchedule(DateTime.Now,
300));
jobService.UpdateJob(jobDefinition);
```

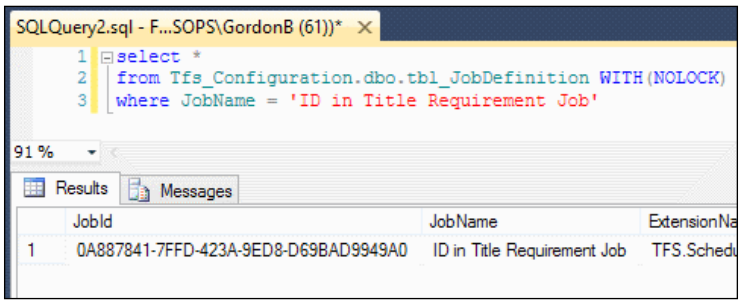
As you can see, the name of our job is `ID in Title Requirement Job` and the full class name is `TFS.ScheduledJobs.IdInTitleRequirementJob`.

Verify that the job is installed

To verify that our job is installed, we can run the following SQL commands against the TFS Configuration database.

```
select * from Tfs_Configuration.dbo.tbl_JobDefinition WITH(NOLOCK)
where JobName = 'ID in Title Requirement Job'
```

This should return a result that contains our newly registered job.



Custom job in the database

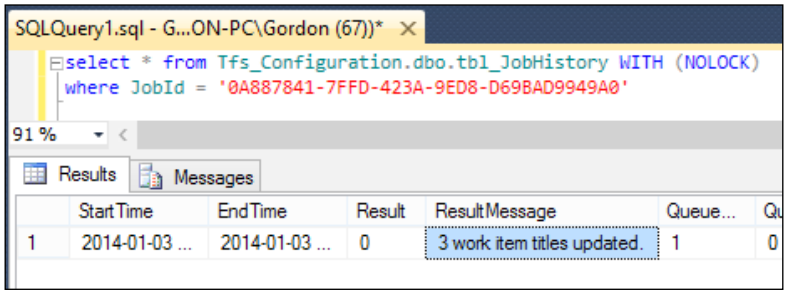
As you can see, our job definition does exist in the database. At this point, you must take note what the `JobId` is so that we can use it again later.

Checking the job history

To check the job history, we will query the TFS Configuration database again. This time, we will look at the `tbl_JobHistory` table, and we will use the `JobId` from the earlier query.

```
select * from Tfs_Configuration.dbo.tbl_JobHistory WITH (NOLOCK)
where JobId = '0A887841-7FFD-423A-9ED8-D69BAD9949A0'
```

This should return an output similar to the following screenshot:



Job history

If you see that your job has a **Result** of **6** (extension not found), then you will need to stop and start the TFS Job Agent. You can do this by running the following commands in an **Administrator CMD** window:

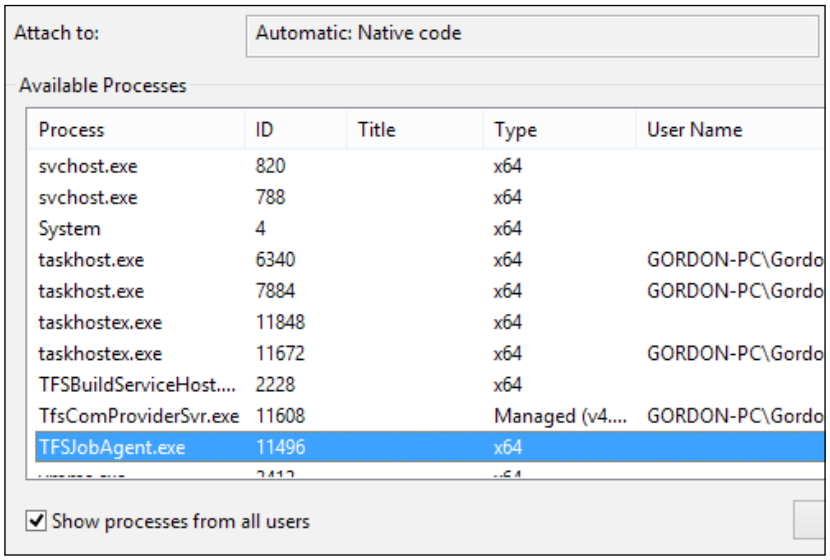
```
net stop TfsJobAgent
net start TfsJobAgent
```

After the agent has started again, you will see that the **Result** field is now different as a job agent that will know about your job.

Another way to view the job history is to browse to your TFS server with the URL `_oi/_jobMonitoring#_a=history` after the site root. For example, `http://gordon-pc:8080/tfs/_oi/_jobMonitoring#_a=history`. On this page, you will see charts that show you the status of all your jobs.

Debugging a TFS job

To debug a TFS job, we must go to **DEBUG | Attach to Process…** on the menu. Then, you need to locate `TFSJobAgent.exe` and attach your Visual Studio to the `TfsJobAgent` process. There should only be one process in the list, and you must make sure you check **Show processes from all users** to show the `TFSJobAgent.exe` process in the list.



Attach to the TFS job agent

The next time your job is run, any break points that are set will be hit as normal and you can debug your code.

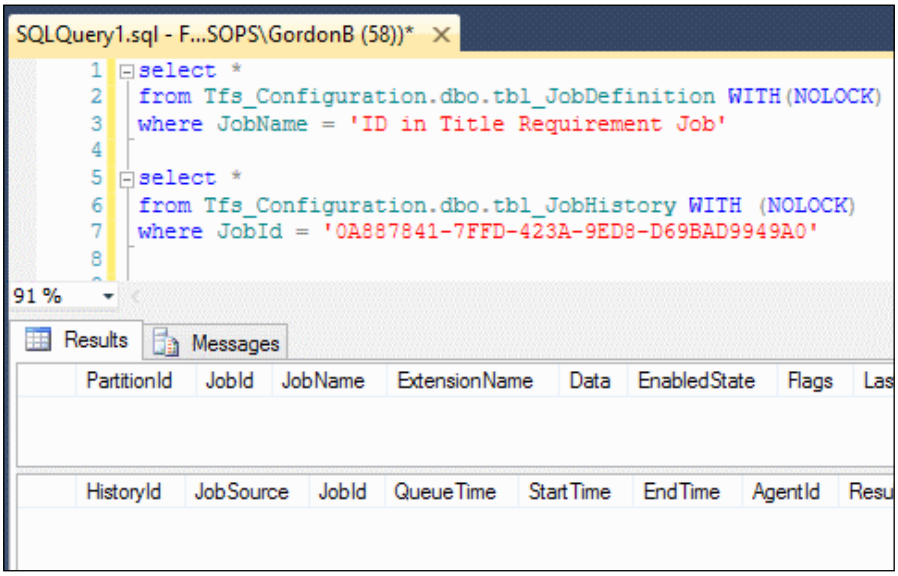
Deregistering the custom job with TFS

To deregister the custom job, we just need to call the `DeleteJob` method on the job service. For this, you will need to have the `JobId` of the job definition.

```
var tfsUri = new Uri("http://localhost:8080/tfs");
var config = TfsConfigurationServerFactory.
GetConfigurationServer(tfsUri);
var jobService = config.GetService<ITeamFoundationJobService>();
jobService.DeleteJob(new Guid("62FDDA25-4938-4BF7-A7C3-
6A9BF527A20C"));
```

After running this, the custom job will be deregistered from the TFS job agent. Because you need the ID of the job to deregister it, you are able to set a static JobId when first registering the job. This way, you will not need to query the jobs to get the `JobId` when you want to remove it.

If you run this code and then re-run the SQL from earlier in this chapter to check the installation and history of our job, you will get two empty results.



SQL results after uninstalling the custom job

As you can see, our TFS job has been successfully removed.

Summary

In this chapter, we went over creating, deploying, registering, and deregistering a TFS job. Also, we briefly looked at what you can try to do if TFS can't find your job after deploying it.

You can read up on permissions, assumptions, and limitations in the MSDN article on Team Foundation Background Job Agent at `http://msdn.microsoft.com/en-us/library/ms252450.aspx`.

In the next chapter, we will be looking at the TFS Build process and how we can improve it, including the ability to integrate Twitter to inform followers on Twitter about a new nightly build for an open source project.

6
Customizing a TFS Build Process

This chapter will demonstrate how to create and use a custom build activity and will cover the following topics:

- Creating a copy of the default build template, which includes:
  - Downloading the default template
  - Using the template in a build
- Creating a custom build activity to share nightly builds with the community, which will consist of the following tasks:
  - Creating the custom activity project
  - Zipping the build output
  - Uploading the ZIP file to FTP
  - Sharing the build number and URL with the ZIP file
- Adding the custom activity to an existing build template:
  - Creating a build template project
  - Adding our custom activity
- Adding build template arguments

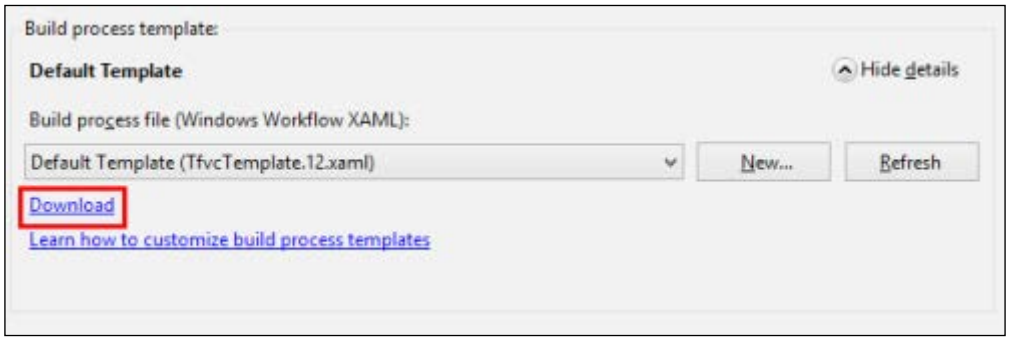
In our example, we will be sharing our build info on Twitter to inform followers where to grab the latest builds for a project.

Creating a copy of the default build template

With TFS 2013, the build templates are not part of a source control by default; so, we will need to get a copy of the default process template, and then add that template to the source control so that we can use it.

Downloading the default template

The first thing we will need to before we can customize a build template is getting the default template and creating a copy of it. To get started, open **Team Explorer**, then click on **Builds**, and then on **New Build Definition**. Select the **Process** step to the left under the **Build process template** section and click on the Show Details arrow. Also, make sure that the **Default Template (TfvcTemplate.12.xaml)** template is selected in the dropdown. Click on the **Download** button as shown in the following screenshot:

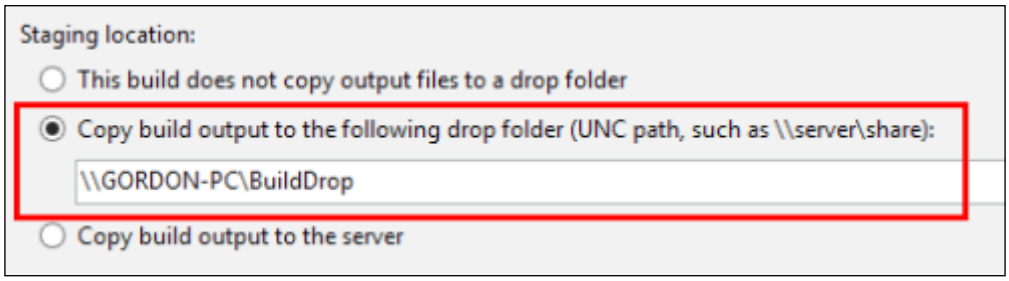


Downloading the build definition

Save the template as `CustomBuildTemplate.xaml`.

Using the template in a build

The next step is to add this template to the source control. Add the template to a folder called `BuildTemplates` in the root directory of a team project and select it. Now, we want to set up a new build, and this build will be used throughout the rest of this chapter when we wish to test our custom build activity. We will use any existing project that we have in the source control for testing our build. Make sure that you copy the build to a `UNC` drop folder.



Copying the build output to the UNC path

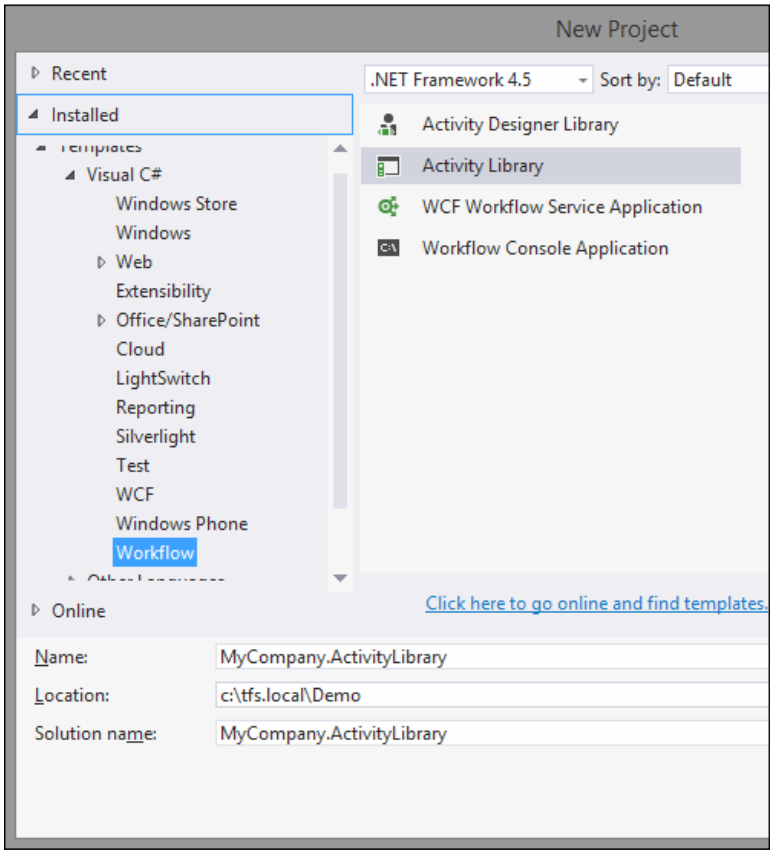
When you get to the **Process** step, click on **Show Details** again and then click on **New**. In the pop up, select your team project and then in **Version control path,** enter `$/<Team Project Name>/BuildTemplates/CustomBuildTemplate.xaml` and click **OK**. Queue a new build, and your build should be built successfully at this point.

Creating a custom build activity

Out of the box, there are many build activities that you can use to customize your build process. If you find this is not enough, you can download and install **TFS Build Extensions** from **Codeplex** (`http://tfsbuildextensions.codeplex.com/`). There will still be scenarios where you need to create your own custom activities. Now, we will be creating a custom activity that will, at the end of our build, zip up our output, publish the ZIP file to the FTP site, and then tweet a link to this file. This sample could be useful for projects where you want to inform your followers of new nightly builds that are available. You can read more about build activities on MSDN at `http://msdn.microsoft.com/en-us/library/gg265783.aspx`.

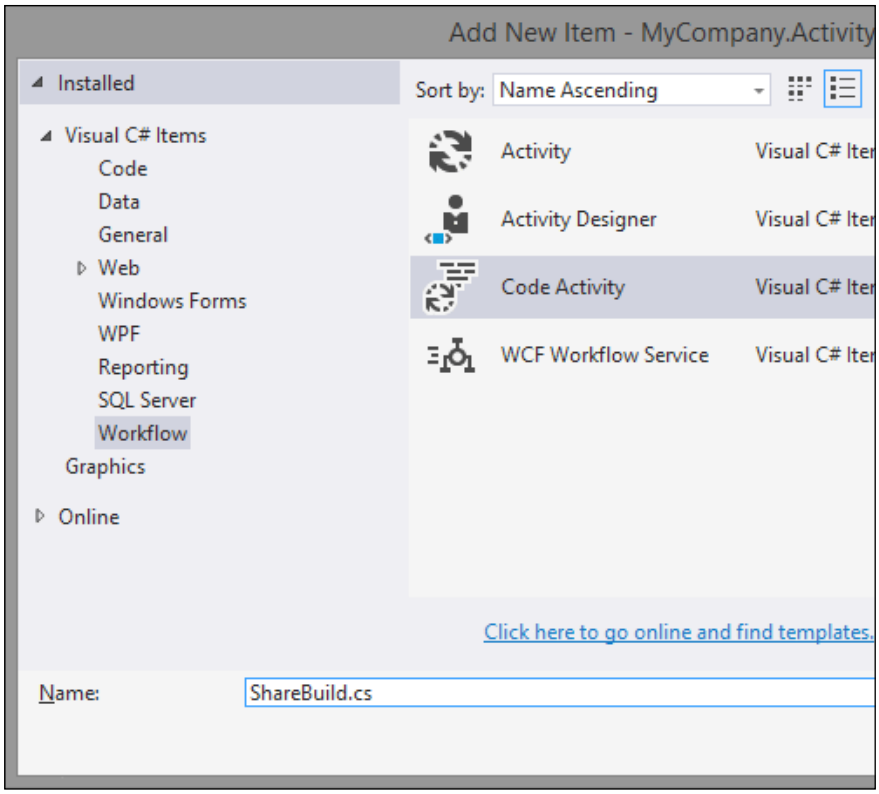
Creating the custom activity project

Create a new C# **Activity Library** project called `MyCompany.ActivityLibrary`. This can be found under the **Workflow** tab in the new project dialog, as shown in the following screenshot:



Creating a new activity library project

Delete the `Activity1.xaml` file that is created for us by default and create a new **Code Activity** called `ShareBuild.cs`, as shown in the following screenshot:



New code activity

Add a reference to `Microsoft.TeamFoundation.Build.Client.dll`, which can be found at `C:\Program Files (x86)\Microsoft Visual Studio 12.0\Common7\IDE\ReferenceAssemblies\v2.0`. Next, you will need to add the following attribute to the `ShareBuild` class:

```
[BuildActivity(HostEnvironmentOption.Agent)]
```

We need to specify the `BuildActivity` attribute so that our activity can be found while scanning our assembly for build activities, and as an agent, `HostEnvironmentOption` informs TFS that this build activity can run only on the agents. This will get us ready to start implementing the logic parts of our custom activity.

Zipping the current build

To zip the current build, we will be using the `ZipFile` class, so you will need to add a reference to the `System.IO.Compression.FileSystem.dll` file. We will also need to get information from the build agent on the current build location in our `Execute` method. We will, for now, just zip up `DropLocation` for the build to the `temp` directory. This can be done using the following code:

```
IBuildDetail buildDetail = context.GetExtension<IBuildDetail>();
string tempFile = Path.GetTempPath() + "\\"+ Guid.NewGuid()+".TMP";
ZipFile.CreateFromDirectory(buildDetail.DropLocation, tempFile);
```

This will create a ZIP file at `tempFile` that we will use later for our FTP.

Uploading the ZIP file to FTP

For the FTP operation, we will need to create an `UploadFile` method, shown as follows:

```
private void UploadFile(CodeActivityContext context, string fileName,
string tempFile)
{
    using (System.Net.WebClient client = new System.Net.WebClient())
    {
        client.Credentials = new NetworkCredential(context.
GetValue(this.FTP_Username), context.GetValue(this.FTP_Password));
        client.UploadFile(context.GetValue(this.FTP_HostName)
+ context.GetValue(FTP_RemoteLocationPath) + fileName, "STOR",
tempFile);
    }
}
```

In order to upload our ZIP file, we will require some values to be set up as part of our build arguments. Add the following properties to the `ShareBuild` class:

```
[RequiredArgument]
public InArgument<string> FTP_HostName { get; set; }

[RequiredArgument]
public InArgument<string> FTP_Username { get; set; }

[RequiredArgument]
public InArgument<string> FTP_Password { get; set; }
```

```
[RequiredArgument]
public InArgument<string> FTP_RemoteLocationPath { get; set; }

[RequiredArgument]
public InArgument<string> FTP_RemoteLocationPathDirectUrl { get; set;
}
```

As you can see, we have used the `RequiredArgument` attribute to specify that we need each of these arguments for our build activity to function properly. If you prefer not to have these inputs, you can remove the properties and instead retrieve the values from another source. You can also use the `InArgument` attribute and then set the direction to either `ArgumentDirection.In`, `ArgumentDirection.Out`, or `ArgumentDirection.InOut`, depending on the usage of the property. However, for our example, we only require the required properties. To perform the upload, we will use the following code:

```
string fileName = buildDetail.BuildNumber.Replace(" ", "-") + ".zip";
UploadFile(context, fileName, tempFile);
```

This will use the credentials and location information provided and FTP the zipped build output file to the FTP location.

Sharing the build number and URL with the ZIP file

For this section, you will need to add a `NuGet` package for **TweetSharp** to your activities project. We will use this library to simplify the way we interact with Twitter. The inputs required for sending a tweet on Twitter are a consumer key, consumer secret key, access token, and access token secret. You can create an application and get all these keys from the Twitter Developer site (`https://dev.twitter.com`). For our custom activity, we will need to provide the following properties to get all these keys:

```
[RequiredArgument]
public InArgument<string> Twitter_ConsumerKey { get; set; }

[RequiredArgument]
public InArgument<string> Twitter_ConsumerSecret { get; set; }

[RequiredArgument]
public InArgument<string> Twitter_AccessToken { get; set; }

[RequiredArgument]
public InArgument<string> Twitter_AccessTokenSecret { get; set; }
```

Now that we have the properties set up for our keys, use the following code to send a status update:

```
var twitterService = new TwitterService(context.GetValue(Twitter_
ConsumerKey), context.GetValue(Twitter_ConsumerSecret), context.
GetValue(Twitter_AccessToken), context.GetValue(Twitter_
AccessTokenSecret));
twitterService.SendTweet(new SendTweetOptions { Status = "[New Bits
Available] " + buildDetail.BuildNumber + " - " + context.GetValue(FTP_
RemoteLocationPathDirectUrl) + fileName });
```

This will get a new instance of the `TwitterService` class using the keys supplied. Then it will send a status update containing the build definition name that is currently being run with the build number and the exact URL to access the file that we zipped and FTP'd.

Adding a custom activity to an existing build template

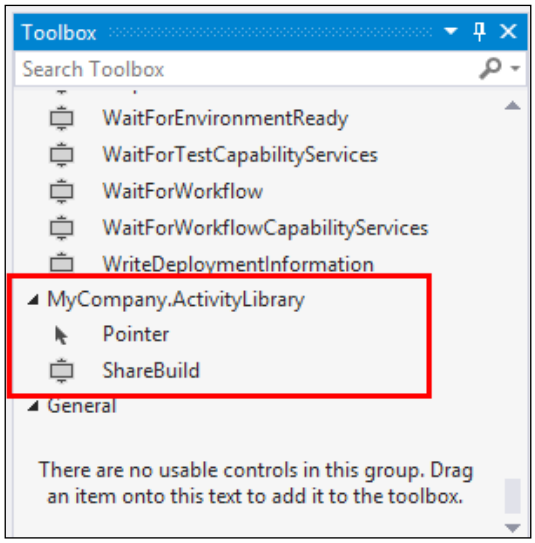
Now that we have created a custom activity, we will need to add it to the build template that we copied earlier.

Creating a build template project

The easiest way to work with build templates is to add the build template to an activity library. Perform the following points to do this:

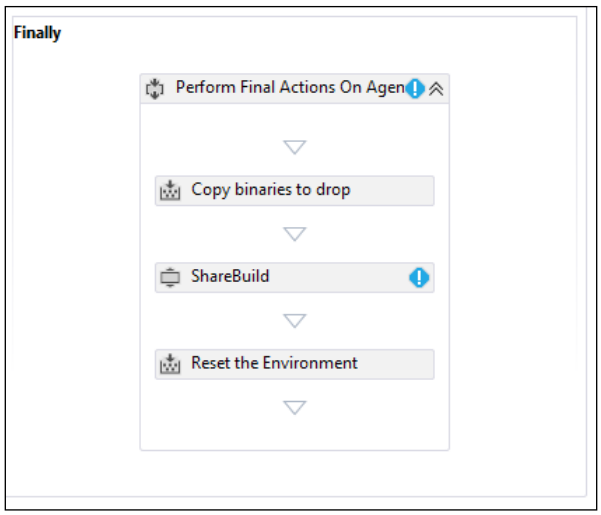
1. Add a new **Activity Library** to the same solution that contains `MyCompany.ActivityLibrary` and call it `MyCompany.CustomBuildTemplates`.
2. Remove the `Activity.xaml` file.
3. To get our template from the earlier project to this project, we will need to add an existing item to this solution, and select the `CustomBuildTemplate.xaml` file from the earlier project.
4. Change the **Build Action** of `CustomBuildTemplate.xaml` to **Content** (make sure you add a link to the item and not just add it, as then we will need to update our build definition to point to a new template).
5. Add a solution reference to **MyCompany.ActivityLibrary**.

If you look in the **Toolbox**, you will now see the `ShareBuild` activity that we created, as shown in the following screenshot:



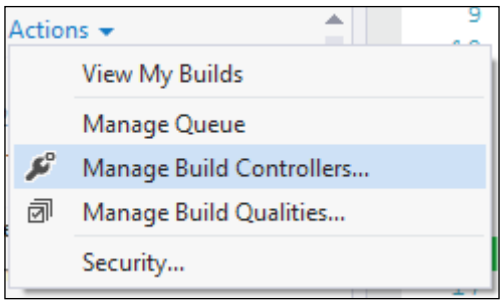
New activity in the toolbox

Open the `CustomBuildTemplate.xaml` template and locate the **Finally** block inside the **Run on Agent** block, which is inside the workflow. Drag the **SharedBuild** activity onto the workflow between the **Copy binaries to drop** and **Reset the Environment** activities, as shown in the following screenshot:



SharedBuild activity added to the workflow

As you can see, there is an exclamation mark (!) next to our activity; this is because we need to set the required properties that we defined in our code earlier. Click on the **ShareBuild** step and then open its properties, configure each of the values in the properties, and click on **Save**. Now that our build template knows about our new custom activity, we need to make TFS aware of it too. Create a `CustomAssemblies` folder in the `BuildTemplates` folder in **Version Control**, take all the output assemblies from `MyCompany.ActivityLibrary`, and place them in this folder. Make sure you check your changes. Next, as shown in the following screenshot, go to the **Team Explorer** | **Builds** window | **Actions** | **Manage Build Controllers…**:



Manage Build Controllers

In the **Manage Build Controllers** dialog, click on **Properties**. Then, set the Version Control path for custom assemblies to the `CustomAssemblies` folder we just created, and click on **OK** twice and then on **Close**. When you set or modify the value in this box, the build server automatically restarts to load the assemblies. You can read more about this on MSDN at `http://msdn.microsoft.com/en-us/library/ee330987.aspx#custom_process`.

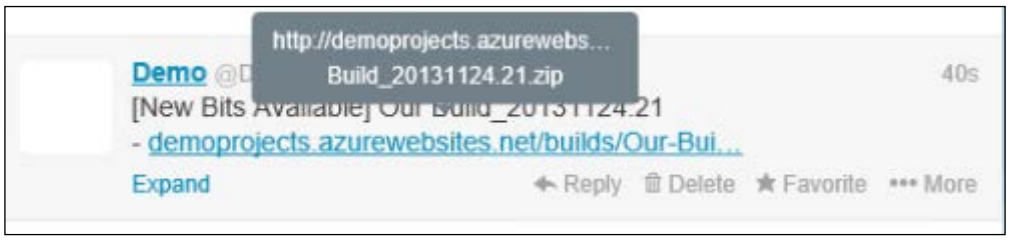
You can now queue a new build, and, as shown in the following screenshot, you will see that it still passes.

Build passed

You will also see that the file exists on the FTP site, as shown in the following screenshot:

The ZIP file on the FTP site

And lastly, you will be able to see that on Twitter, our Twitter account is updated with our status containing information on where to get the new release, shown as follows:

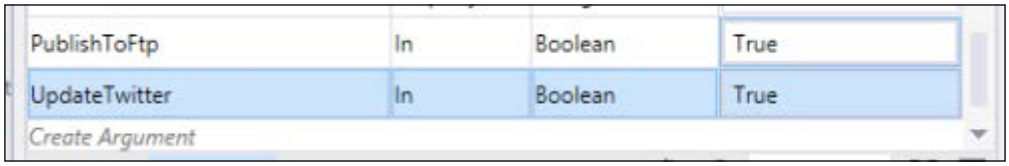


The Twitter update

As you can see, we have successfully created a new custom activity and configured one of our build templates to use it as part of the regular build.

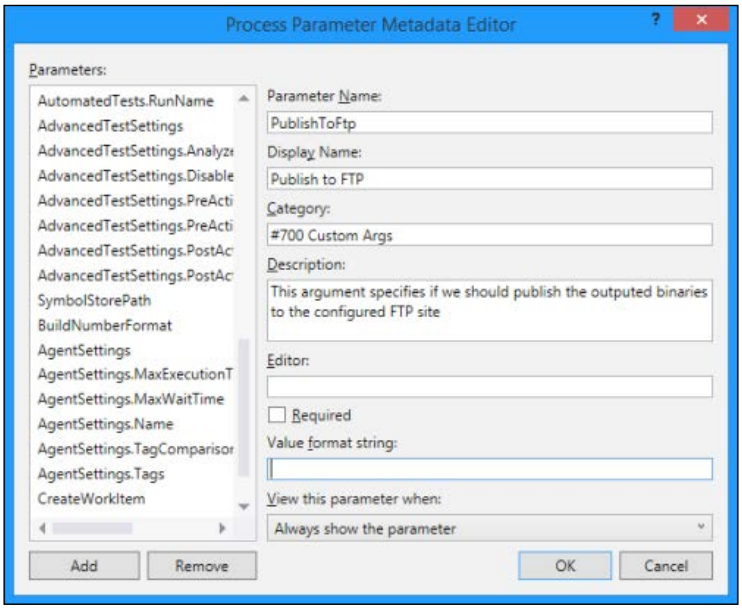
Adding a build template argument

Sometimes you won't always want to run the publishing to FTP, and also when you do publish, you might not want to post an update on Twitter. Instead of creating multiple templates that can cater to different processes, we are able to add build template arguments. To get started with this, open up the `CustomBuildTemplate.xaml` file and click on the arguments located below the **Workflow Designer**. Scroll to the bottom of the grid and add two Boolean arguments for **PublishToFtp** and **UpdateTwitter**, shown as follows:



Creating new arguments

Now that we have added our arguments, we want to customize the display name of our arguments and place them in a common build property category. Find the **Metadata** argument in the list, and click on the button to open the collection of metadata. Click on **Add** and configure the values as shown in the following screenshot:



The Process Parameter Metadata Editor

Now, configure another field by clicking on **Add** and use the values below **Parameter Name**: UpdateTwitter, **Display Name**: Update Twitter, **Category**: #700 Custom Args, and a **Description**: This argument specifies if we should update Twitter during our build.

Note that if **Publish to FTP** is false, this will be skipped. Click on **OK**. Next, we will use these values in our custom activity to allow the skipping of part or all of our activity. Add the following two properties to the `ShareBuild` class:

```
[RequiredArgument]
public InArgument<bool> Config_PublishToFtp { get; set; }
[RequiredArgument]
public InArgument<bool> Config_UpdateTwitter { get; set; }
```

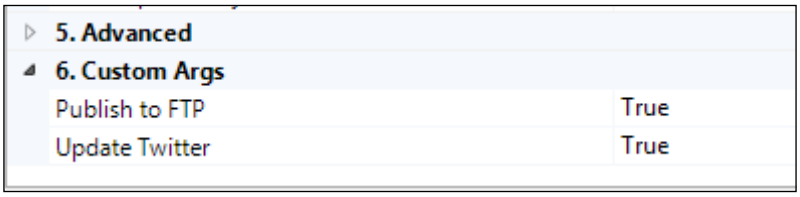
And then modify the code in the `Execute` block with the following `if` statement:

```
if (context.GetValue(Config_PublishToFtp))
{
    // Execute block of code in here
}
```

Then finally, wrap the Twitter block of code with the following code:

```
if (context.GetValue(Config_UpdateTwitter))
{
    // Twitter block of code in here
}
```

Build and check in the updated custom activity and the updated build template, and you will then see, as shown in the following screenshot, that when you edit your build definition, there is a new category of arguments called **Custom Args**. You will also see that when you queue a new build, you are able to select the arguments per build.



New args in the build definition argument list

The last step to tie our argument list to our code is to edit the properties of our **ShareBuild** activity inside the workflow. Set the `Config_PublishToFtp` property to have a value of `PublishToFtp` and `Config_UpdateTwitter` to have a value of `UpdateTwitter`. You will now see that if you set these values to `false`, you will have a regular build, and nothing will be published to FTP or interact with Twitter.

Summary

In this chapter, we customized an existing TFS build template by adding the ability to publish the built binaries to an FTP site and then update our Twitter status to inform our followers about the new bits. We then looked at how we can add in new arguments so that we don't need to create multiple build templates for the different processes that we would want to follow when there are minor differences between them.

To find out more about build customization, you can view the *ALM Rangers Team Foundation Build Customization guide* on **Codeplex** at `http://vsarbuildguide.codeplex.com/`.

Index

A

activity project
  creating 72, 73
add panels
  fields, adding to 25, 26
Administer Server button 28

B

Backlog items query 21
browse button 41
build activity
  activity project, creating 72, 73
  build number, sharing to ZIP file 75, 76
  creating 71
  current build, zipping 74
  URL, sharing to ZIP file 75, 76
  ZIP file, uploading to FTP 74, 75
build definitions
  pinning, to team dashboard 22, 23
build number
  sharing, to ZIP file 75, 76
build template
  argument, adding 79-81
  custom activity, adding to 76-79
  project, creating 76-79

C

change event 43
check-in plugin
  creating 54-56
check-in policies
  using 51
  versus, TFS plugin 51
CI (continuous integration) builds 22
class library 38
Clear method 39
Codeplex 71
Config_PublishToFtp property 81
control
  using, on existing WITD 44, 45
Control Settings property 46
current build
  zipping 74
custom activity
  adding to, build template 76-79
custom job
  deregistering, TFS used 67

D

dashboard
  recent changes, pinning to 23, 24
  tile, adding to 20, 21
decision point 55
default build template
  copy, creating 70
  downloading 70
  using 71
default columns
  changing, for portfolio boards 24, 25
DeleteJob method 67
Developer CMD console 13
display work items
  information, adding to 31-34
Download button 70

E

Epic Work Item Type 14
Execute method 74

F

Fiddler AutoResponder rule
configuring 30, 31
field
adding, raw .xml files used 11, 12
adding, TFS Power Tools used 6-10
adding, to add panels 25, 26
adding, to work item form layout with TFS Power Tools 10
Fields Definition window 7
FlushToDatasource method 39
FTP
ZIP file, uploading to 74, 75
FullClassName class 41

I

IdInTitleRequirementJob class 62
IIS 53
InArgument attribute 75
initialize function 28, 29
Internet Information Services. *See* **IIS**
InvalidateDatasource method 39

J

JavaScript plugin
creating 26-28
debug mode, setting up 29
importing, into TFS Web Access 28, 29
information, adding to display work items 32-34

L

loadAfter attribute 42

M

Manage Build Controllers dialog 78
Microsoft Visual Studio Scrum 2013 process template 5
MyCompany.WebAccess.Plugin namespace 27

O

onComplete function 33

P

portfolio
adding, to process template 14
portfolio backlog
adding 14, 15
portfolio boards
default columns, changing for 24, 25
ProcessEvent method 58
process template
portfolio, adding to 14
portfolio backlog, adding 14, 15
work item category, adding 15
work item portfolio, importing 15, 16
work item type definition, creating 14
Product Backlog Item button 24

R

raw .xml files
used, for adding work item form layout field 12
used, for altering work item workflow 12
used, for field adding 11, 12
used, for WITD importing 13
recent changes
pinning, to dashboard 23, 24
RequiredArgument attribute 75
RequireJs module loader 27
Restricted options field 6
resultMessage variable 64
rich data web page
displaying, inside WITD 46-48

S

Save() method 59
Script Debug Mode
enabling 30
server plugin
code, setting up 52
creating 52-54
debugging 53, 54
deploying 53

setInterval function 32
ShareBuild class 73, 74, 80
SharedBuild activity 77
Stream of Work 6
SubscribedTypes method 54, 57

T

team dashboard
  build definitions, pinning to 22, 23
Text Field 6
TF237079 error 62
TFS
  build definitions, pinning to team dashboard 22, 23
  dashboard tiles, customizing 19
  recent changes, pinning to dashboard 23, 24
  tile, adding to dashboard 20, 21
  used, for custom job deregistering 67
TFS Background Job Agent 61
TFS Build Extensions
  URL 71
TfsCheckbox 45
TFS job
  adding, to schedule 64
  code, deploying 64
  code, writing 62, 64
  creating 62
  debugging 66
  installation, verifying 65
  registering 64
TFS job history
  checking 65
TFS plugin
  versus, check-in policies 51
TFS Power Tools
  installing, URL 5
  used, for adding field 6-10
  used, for adding work item form layout field 10
  used, for WITD importing 11
TFS Scrum template 14
TFS Web Access
  JavaScript plugin, importing to 28, 29
TFS Web Access interface
  customizing 19
TFS work items
  customizing 6
tile
  adding, to dashboard 20, 21
TweetSharp 75
TwitterService class 76
Type property 46
TypeScript
  using 27

U

UploadFile method 74

V

View queries link 21

W

Web Access control
  code, debugging 43
  code, deploying 43
  code, writing 41-43
  creating 41
Windows Forms control
  code, debugging 41
  code, deploying 40
  code, writing 38-40
  creating 38
witadmin importprocessconfig tool 24
witadmin importwitd tool 45
witadmin tool 13
WITD
  about 6, 44
  control, using on 44, 45
  creating 14
  importing, raw .xml files used 13
  rich data web page, displaying inside 46-48
work item category
  adding 15
work item changed plugin
  creating 57-59
work item form layout field
  adding, raw .xml files used 12
  adding, TFS Power Tools used 10
work item portfolio
  categories file, importing 16

importing 15
process configuration file, importing 16
work item type definition, importing 16
Work Item Type Definition. *See* **WITD**
work item workflow
altering, raw .xml files used 12

Z

ZIP file
build number, sharing to 75, 76
uploading, to FTP 74, 75
URL, sharing to 75, 76

[PACKT] PUBLISHING enterprise professional expertise distilled

Thank you for buying

Team Foundation Server 2013 Customization

About Packt Publishing

Packt, pronounced 'packed', published its first book "Mastering phpMyAdmin for Effective MySQL Management" in April 2004 and subsequently continued to specialize in publishing highly focused books on specific technologies and solutions.

Our books and publications share the experiences of your fellow IT professionals in adapting and customizing today's systems, applications, and frameworks. Our solution based books give you the knowledge and power to customize the software and technologies you're using to get the job done. Packt books are more specific and less general than the IT books you have seen in the past. Our unique business model allows us to bring you more focused information, giving you more of what you need to know, and less of what you don't.

Packt is a modern, yet unique publishing company, which focuses on producing quality, cutting-edge books for communities of developers, administrators, and newbies alike. For more information, please visit our website: `www.packtpub.com`.

About Packt Enterprise

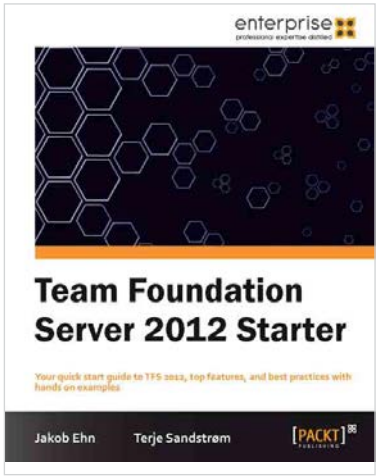
In 2010, Packt launched two new brands, Packt Enterprise and Packt Open Source, in order to continue its focus on specialization. This book is part of the Packt Enterprise brand, home to books published on enterprise software – software created by major vendors, including (but not limited to) IBM, Microsoft and Oracle, often for use in other corporations. Its titles will offer information relevant to a range of users of this software, including administrators, developers, architects, and end users.

Writing for Packt

We welcome all inquiries from people who are interested in authoring. Book proposals should be sent to `author@packtpub.com`. If your book idea is still at an early stage and you would like to discuss it first before writing a formal book proposal, contact us; one of our commissioning editors will get in touch with you.

We're not just looking for published authors; if you have strong technical skills but no writing experience, our experienced editors can help you develop a writing career, or simply get some additional reward for your expertise.

[PACKT] PUBLISHING enterprise professional expertise distilled

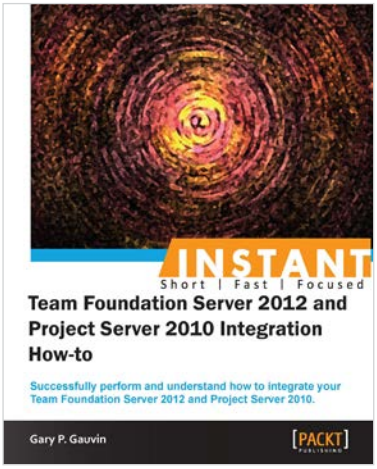


Team Foundation Server 2012 Starter

ISBN: 978-1-84968-838-3 eBook: 72 pages

Your quick start guide to TFS 2012, top features, and best practices with hands on examples

1. Learn something new in an Instant! A short, fast, focused guide delivering immediate results
2. Install TFS 2012 from scratch
3. Get up and running with your first project



Instant Team Foundation Server 2012 and Project Server 2010 Integration How-to

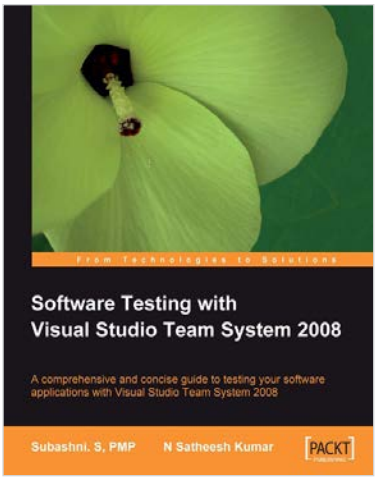
ISBN: 978-1-84968-854-3 eBook: 54 pages

Successfully perform and understand how to integrate your Team Foundation Server 2012 and Project Server 2010.

1. Learn something new in an Instant! A short, fast, focused guide delivering immediate results
2. Learn to plan and successfully implement your Team Foundation Server and Project Server integration
3. Easily install or upgrade your Team Foundation Server extensions for Project Server

Please check **www.PacktPub.com** for information on our titles

[PACKT] PUBLISHING enterprise
professional expertise distilled



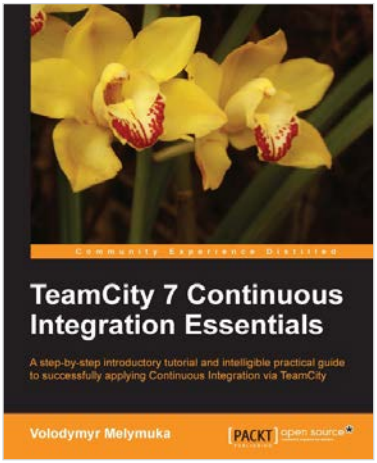
Software Testing with Visual Studio Team System 2008

ISBN: 978-1-84719-558-6 Paperback: 356 pages

A comprehensive and concise guide to testing your software applications with Visual Studio Team System 2008

1. Test your software applications with Visual Studio Team System 2008 and rest assured of its quality
2. Create a structured testing environment for your applications to produce reliable products
3. Comprehensive yet concise guide with a lot of examples and clear explanations

---



TeamCity 7 Continuous Integration Essentials

ISBN: 978-1-84969-376-9 Paperback: 128 pages

A step-by-step introductory tutorial and intelligible practical guide to successfully applying Continuous Integration via TeamCity

1. Put Continuous Integration into operation with TeamCity, quickly and easily with this practical tutorial
2. Set automatic build checks and notifications according to your needs and configure multistep builds with dependent and interrelated projects easily
3. Plug TeamCity either to existing on-going development or at the project's very beginning

Please check **www.PacktPub.com** for information on our titles